

**Phishing URL Detection on PhiUSIIL: A Leakage-Aware Comparison of
Classical, Character-Level, and Clustering Methods**

Pranav Dhinakar, Jason Justice, and and Guna Pasupathy

Shiley-Marcos School of Engineering

University of San Diego

AAI-501: Introduction to Artificial Intelligence

Professor Andrew Van Benschoten

August 10, 2026

Author Note

Group 2 final team project. Source code and full commit history are available in the team GitHub repository:

<https://github.com/jjusticeusd/AAI-501-Final-Team-Project-Group2.git>. A disclosure of generative-AI assistance appears in Appendix B.

Abstract

Phishing remains one of the most common vectors for scams and data breaches, and automated URL classification is a practical application of supervised learning. We build and evaluate a phishing-detection pipeline on the PhiUSIIL Phishing URL Dataset (235,795 labeled URLs, 54 features) and organize the work around a single finding: on this benchmark, near-perfect separability is intrinsic to the data rather than an artifact of any one leaky column or train/test split. We audit the dataset for leakage and confirm that content and lexical features reach roughly 99.9% accuracy with or without the obviously leaky `URLSimilarityIndex`. We then compare three classical classifiers (logistic regression, random forest, and XGBoost) with class-imbalance, tuning, feature-importance, and cost-sensitive analyses; a character-level convolutional network trained on the raw URL string alone as a leakage-immune “hard mode” comparison; and an unsupervised clustering of phishing rows into interpretable attack families. The character model reaches 0.9985 accuracy and 0.9993 PR-AUC using only URL characters, essentially tying the full-feature tabular models and demonstrating that the separability lives in the URL strings themselves. All supervised models land inside the published 99.5–100% PhiUSIIL band. We therefore frame the contribution not as a new accuracy ceiling but as an explanation of why the ceiling exists and what kinds of phishing the benchmark contains.

Phishing URL Detection on PhiUSIIL: A Leakage-Aware Comparison of Classical, Character-Level, and Clustering Methods

Introduction

Phishing is among the most common techniques used for scams and data breaches, and automated detection of malicious web addresses is a practical, high-impact application of supervised machine learning. This project builds and evaluates a system that classifies a URL as phishing or legitimate and, in doing so, compares more than one family of algorithms under a controlled experimental design, as the course project requires.

We use the PhiUSIIL Phishing URL Dataset (UCI Machine Learning Repository, id 967), which contains 235,795 labeled URLs described by 54 features. The target column `label` encodes 1 for legitimate and 0 for phishing. The dataset is large, has no missing values, and ships with pre-engineered lexical and page-content features, so it satisfies the project's size and preprocessing constraints without heavy cleaning.

The intellectual core of the project is a leakage-first framing. PhiUSIIL is known to be “too easy” when every column is handed to a model, and a naive report could stop at a near-perfect accuracy number. We instead treat that near-perfection as the thing to be explained. Our central finding, established three separate ways across the notebooks, is that the separability of phishing from legitimate URLs is a property of the feature set and even of the raw URL strings, not a side effect of one leaky column or a favorable split. Every modeling result in this report is read through that lens.

The work maps onto several topics from the course. It uses supervised binary classification and ensemble methods (logistic regression, random forest, and gradient boosting), feature selection and model-based feature importance, and model evaluation under mild class imbalance (precision, recall, F1, ROC-AUC, and PR-AUC). It connects to deep learning through a character-level convolutional network applied to raw URL text, which also touches the NLP idea of learning from character sequences. Finally it uses unsupervised clustering (KMeans and HDBSCAN) to characterize structure within the

phishing class, satisfying the requirement to combine at least two different algorithm types.

The report follows the three notebooks that make up the project code. Section 2 presents the data audit and exploratory analysis (Notebook 01, Guna Pasupathy), which establishes the leakage-clean feature set and split conventions used everywhere else. Section 3 presents the classical models, class-imbalance handling, tuning, feature importance, and a cost-sensitive decision layer (Notebook 02, Jason Justice). Section 4 presents the grouping decision, the character-level model, the clustering of attack families, and the state-of-the-art comparison (Notebook 03, Pranav Dhinakar). Section 5 synthesizes the three notebooks against the leakage thesis and the project requirements, and Section 6 concludes.

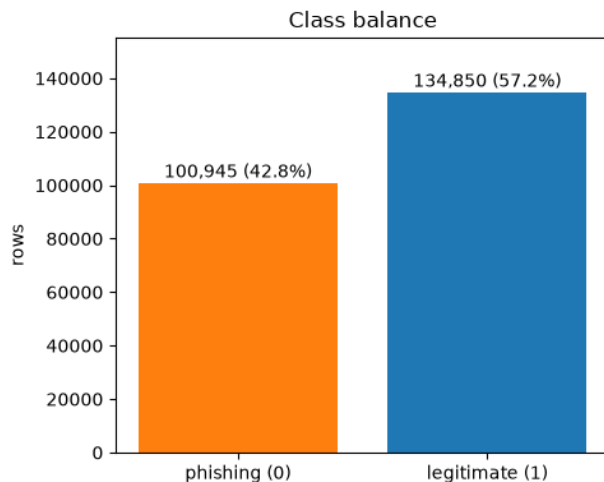
Data, Leakage Audit, and Exploratory Analysis

This section reproduces the analysis and commentary of Notebook 01 (author: Guna Pasupathy). The dataset is the PhiUSIIL Phishing URL Dataset (UCI id 967): 235,795 URLs, 54 features, with `label` equal to 1 for legitimate and 0 for phishing. Because this dataset is known to be “too easy” if every column is thrown at a model, the notebook checks for data leakage before any modeling: it loads the data, looks at class balance and duplicates, checks the leaky and suspicious columns (`URLSimilarityIndex`, too-perfect yes/no features, repeated domains, `TLD`), and saves the list of safe feature columns for Notebooks 02 and 03.

There are no missing values at all, which matches the UCI description. The four string columns are `URL`, `Domain`, `TLD`, and `Title`.

Class Balance

About 57% of the rows are legitimate and 43% are phishing (Figure 1). The proposal feedback assumed a heavily imbalanced dataset, but this is only a mild imbalance, so aggressive resampling is not needed. The plan is to use class weights in the models and to try SMOTE only as a comparison experiment (Notebook 02).

**Figure 1**

Class balance in PhiUSIIL. Legitimate URLs (label 1) slightly outnumber phishing URLs (label 0), a roughly 57/43 split.

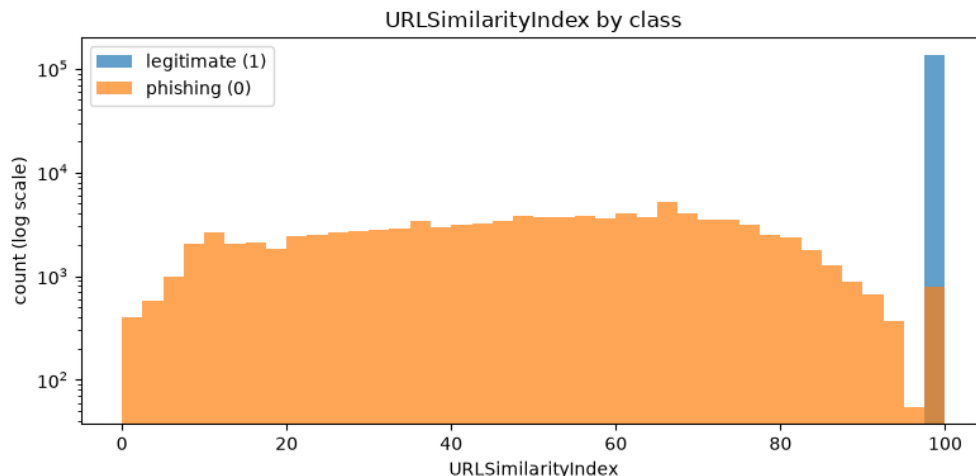
Duplicate Rows and Duplicate URLs

There are no fully duplicated rows, but 425 URLs appear more than once (always with the same label). If the same URL lands in both train and test, the model gets tested on something it literally saw during training. Notebook 02 therefore drops duplicate URLs before splitting.

Leakage Check 1: URLSimilarityIndex

This column measures how similar a URL is to known legitimate URLs, so it basically contains the answer. Every single legitimate URL scores exactly 100 (standard deviation 0), which is clearly leakage. It is not perfect in the other direction though: 786 phishing URLs also score 100, so the column alone cannot reach 100% accuracy (Figure 2). The log scale is needed because the legitimate spike at 100 (134,850 rows) would otherwise flatten the phishing distribution.

How much does this one column matter? A quick logistic regression shows the leaky column alone gets 99.6%. But here is the surprise: dropping it barely matters, because the remaining roughly 50 features together get 99.9%. So the dataset is almost trivially easy even without the leaky column. That makes sense (features like `HasPasswordField` or

**Figure 2**

Distribution of `URLSimilarityIndex` by class (log-scaled counts). Legitimate URLs pile up at exactly 100; a minority of phishing URLs also reach 100.

`LineOfCode` come from crawling the actual page), but it means the tabular models solve an easy version of the problem. That is why the plan also includes a model that only sees the raw URL string (Notebook 03), which is the honest “hard mode” result.

Leakage Check 2: Too-Perfect Yes/No Features

If one class almost always has a flag and the other almost never does, that smells like an artifact of how the dataset was collected rather than a real-world rule. The standout is `IsHTTPS`: 100% of legitimate URLs use HTTPS, with zero exceptions, while about half the phishing URLs do too. A rule “no HTTPS = phishing” would never be wrong on this dataset, which is not true of the real web; it is a side effect of where the legitimate URLs were collected. `HasSocialNet`, `HasCopyrightInfo`, and `HasDescription` have huge gaps too (real sites have footers and meta tags, parked phishing pages usually do not). Dropping the top six giveaway flags (including `IsHTTPS`) and re-running the quick model barely changes accuracy. The easiness is spread across the whole feature set, not hidden in a few columns, so there is no clean line where we could keep “some” content features and get an honest problem. Decision: keep these features for the tabular models (Notebook 02) but call out `IsHTTPS` and friends in the report as collection artifacts, and

treat the URL-only model in Notebook 03 as the realistic difficulty level.

Leakage Check 3: Repeated Domains

URL and Domain are basically IDs, so they stay out of the feature set either way. But if the same domain shows up in both train and test, the model can memorize the domain instead of learning patterns. About 9% of rows share a domain with another row. The top of the list is telling: `ipfs.io`, `gateway.ipfs.io`, `cloudflare-ipfs.com` and `cf-ipfs.com` are all gateways to the same IPFS service, so exact string matching actually *under-counts* how related the rows are. Being stricter would mean grouping by registered domain (for example with the `tldextract` package), which Notebook 03 revisits. Comparing a normal random split against a split that keeps each domain on one side only shows essentially no difference: the score was not inflated by repeated domains, the dataset is just this easy. Still, the grouped split is the safer choice and costs nothing, so Notebook 02 uses `GroupShuffleSplit` on `Domain` and reports both.

The TLD Column

Two findings. First, 695 unique TLDs is too many to one-hot encode. Second, 599 of them are not TLDs at all: for URLs that use a raw IP address, whoever built the dataset took the last number of the IP as the “TLD” (for example, domain `198.98.58.123` gets TLD `123`). All 599 of those rows are phishing. `IsDomainIP` already captures this signal, so it is a data-quality quirk to mention, not something to fix. Instead of encoding TLD the plan is to use `TLDLegitimateProb`, a provided score for how trustworthy the TLD is. One worry: if UCI computed that score *from this dataset’s own labels*, it would be target leakage just like `URLSimilarityIndex`. If the score had been computed from these labels, its correlation with the per-TLD share of legitimate rows would be close to 1; at about 0.18 it clearly was not, so it is fine to keep as a feature.

Which Safe Features Correlate with the Label?

The strongest safe features are mostly page-content ones (`HasSocialNet`, `HasCopyrightInfo`, `IsHTTPS`, and so on), matching the leakage-check results above

(Figure 3). One caveat: plain correlation understates skewed count features like `LineOfCode` or `NoOfImage`, so the notebook relies on model-based feature importance in Notebook 02 rather than this chart for feature selection.

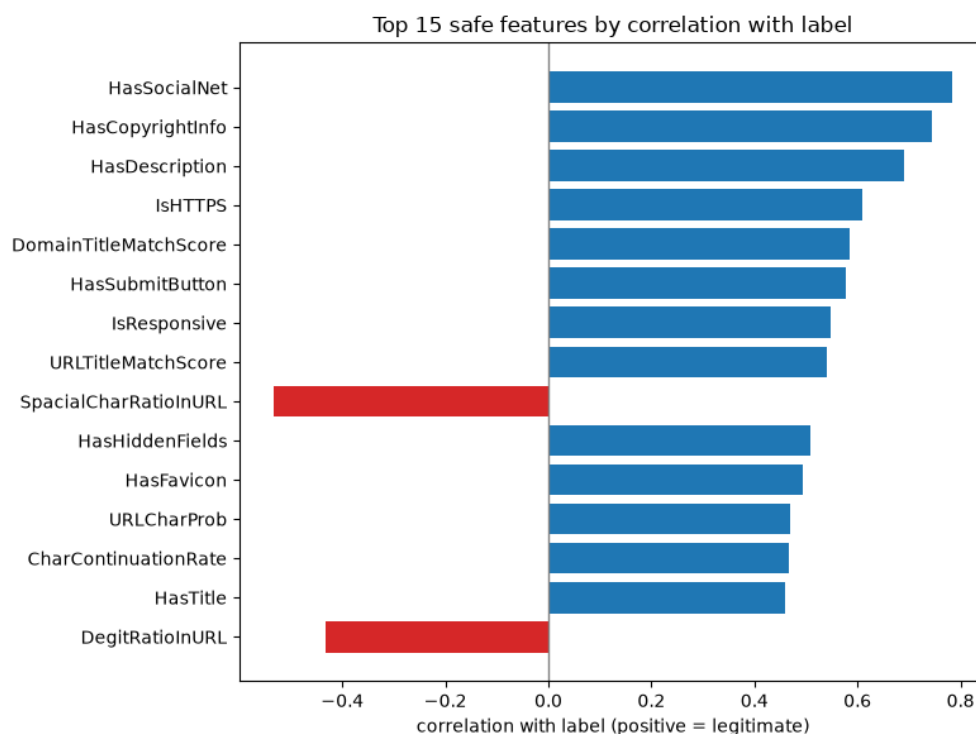


Figure 3

Top 15 safe features by absolute correlation with the label. Page-content indicators dominate; blue is positive and red is negative correlation.

A Closer Look at Individual Features

Everything above was about leakage. A normal EDA question is how the features actually differ between the two classes. The page-content features are the dramatic ones: the median phishing page has 12 lines of code, zero JS files, zero images, and zero external references, while the median legitimate page has over 1,100 lines of code. Most phishing pages are basically empty shells. The URL-shape differences are real but much smaller: phishing URLs are a bit longer and use more special characters. Figure 4 shows distributions for a few representative features (top row = URL features, bottom row = page-content features). The content features (bottom row) separate the classes almost by

themselves, with phishing piling up at zero, while the URL-shape features (top row) overlap a lot. This is another preview of why the URL-only model in Notebook 03 faces the harder problem.

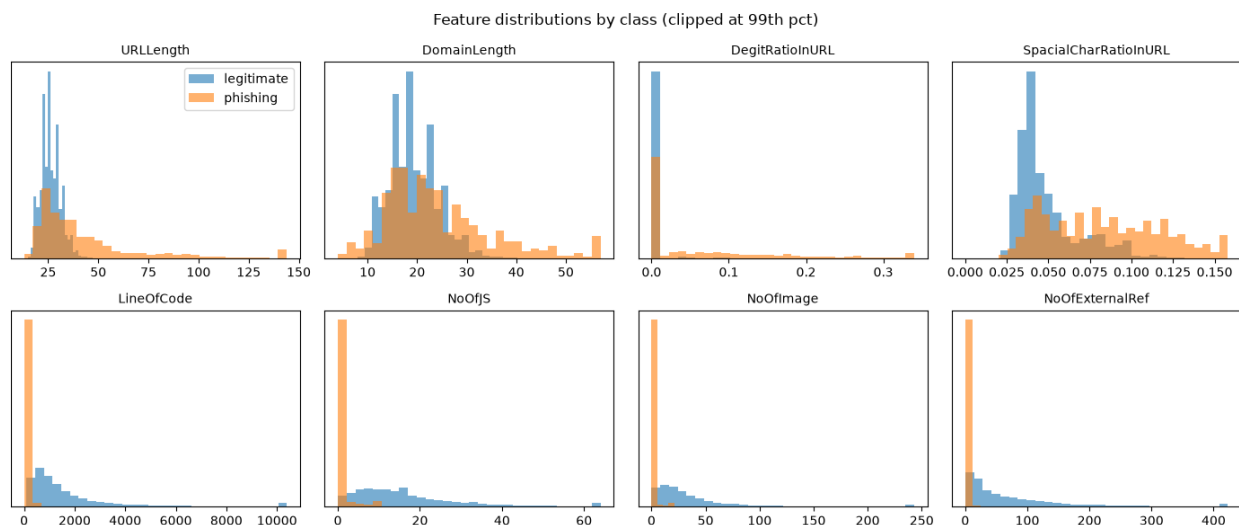
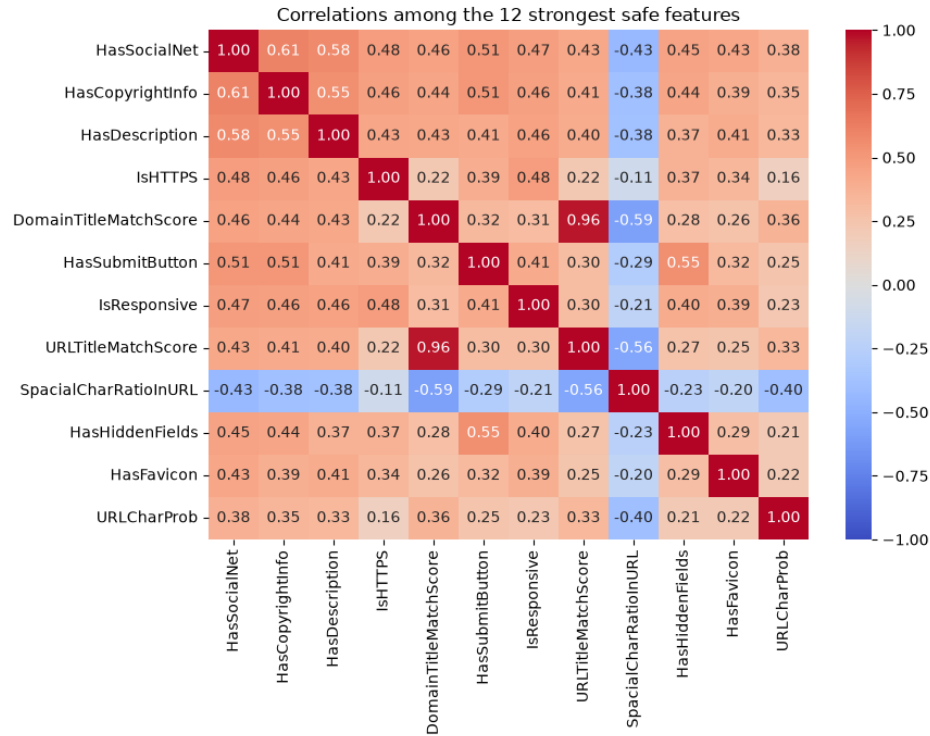


Figure 4

Per-class distributions (clipped at the 99th percentile) for representative URL-shape features (top) and page-content features (bottom). Phishing pages concentrate at zero on the content features.

Do Any Features Duplicate Each Other?

A lot of columns look like counts and ratios of the same thing. Strongly correlated features make logistic-regression coefficients hard to read and are a known risk for the SMOTE experiment. There is less redundancy than expected: only four pairs above an absolute correlation of 0.8, and only two near-duplicates (`DomainTitleMatchScore` / `URLTitleMatchScore` and `URLLength` / `NoOfLettersInURL`). Nothing so extreme that features need to be dropped up front. The heatmap (Figure 5) does show the strongest features are moderately correlated with each other; they all measure some version of “does this page have real content.” So when Notebook 02 looks at feature importance, credit may be split between similar features and exact rankings should not be over-interpreted.

**Figure 5**

Correlation heatmap among the twelve features most correlated with the label. The strong content features are moderately inter-correlated.

Summary of the Data Audit

The audit establishes the following, all of which shape the modeling in Notebooks 02 and 03:

- **Class balance is mild** (57/43), not severe. Class weights first, SMOTE only as a comparison.
- **URLSimilarityIndex is leaky** (every legitimate row equals 100) and is dropped, but the other features still reach roughly 99.9%, so the dataset is easy with or without it. The URL-string-only model in Notebook 03 is the realistic-difficulty comparison.
- **IsHTTPS is a collection artifact** (100% of legitimate rows are HTTPS). It is kept but flagged; dropping the top giveaway flags barely changes accuracy.
- **425 duplicate URLs** lead Notebook 02 to drop URL duplicates before splitting.

- **9% of rows share a domain**, so Notebook 02 uses a domain-grouped split. The quick check says it does not change the score, but it is the safer setup.
- **599 IP-address rows have junk numeric “TLDs”** (all phishing), a data-quality quirk worth a sentence in the report.
- **TLDLegitimateProb checks out as non-leaky** (correlation about 0.18 with the labels), so it is used instead of one-hot encoding 695 TLDs.
- **Phishing pages are empty shells**: median 12 lines of code and zero JS/images/external references, versus 1,105 lines of code for the median legitimate page. URL-shape features overlap far more between the classes.
- **Feature redundancy is mild** (only four pairs with absolute correlation above 0.8), but the strong content features are moderately inter-correlated, so feature-importance rankings in Notebook 02 should not be over-interpreted.

The safe feature list is saved to `data/01_feature_columns.json` so the other notebooks all use the same agreed feature set.

Classical Models

*This section reproduces the modeling and commentary of Notebook 02 (author: Jason Justice). Building on Notebook 01’s leakage-clean feature list, it (1) engineers new features from the raw URL/Domain strings, (2) trains three classifiers on the clean feature set, (3) handles the mild class imbalance and tests whether SMOTE helps, (4) tunes the strongest model and reads off feature importance, (5) checks whether a handful of top features match the full model, and (6) adds a cost-sensitive decision layer. An important note on label orientation: the raw `label` is 1 for legitimate and 0 for phishing, and this notebook models **phishing as the positive class** ($y = 1 - \text{label}$) so that precision, recall, and PR-AUC all describe *catching phishing*.*

Engineered Features

PhiUSIIL already ships lexical ratios and page-content counts. To add something genuinely new, the notebook builds four signals from the raw URL / Domain strings (held out of the clean set as identifiers) that those columns do not already encode:

- **url_entropy / domain_entropy**: character-level Shannon entropy. Randomly generated (DGA-style) domains tend to look more random.
- **brand_distance**: the smallest normalized edit distance from any significant domain label to a curated brand list. A domain that is *close but not equal* to a known brand is the classic typosquatting signal (for example, `paypa1`, `paypal-secure`).
- **is_punycode**: the domain uses xn- (IDN homograph attacks).
- **is_shortener**: the registered domain is a known URL shortener.

A quick look before modeling (medians by class and the distributions of the three continuous features) appears in Figure 6.

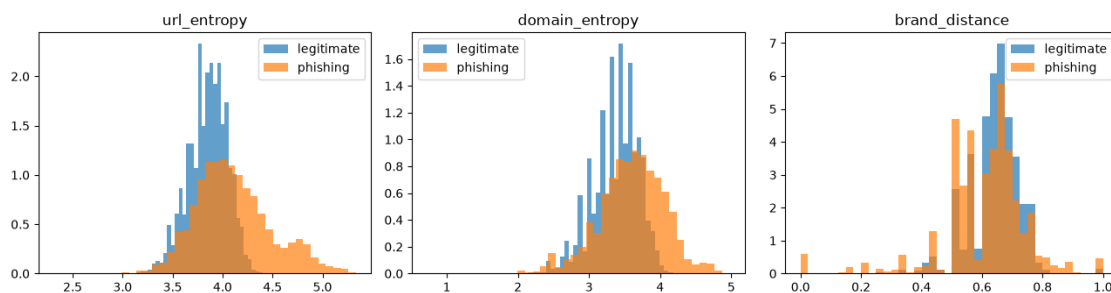


Figure 6

Distributions of the engineered continuous features by class. Class separation is modest, foreshadowing their low importance once the shipped content features are present.

Assembling the Modeling Frame

Following Notebook 01's manifest, the notebook drops duplicate URLs (so the same URL cannot land in both train and test), models phishing as the positive class, and splits with a domain-grouped 80/20 split so every row of a domain stays on one side.

Model Roster

Three classifiers are used, all treating phishing as the positive class: logistic regression (class-weighted) as an interpretable linear baseline; random forest (class-weighted) for a non-linear model with feature importance; and XGBoost (`scale_pos_weight`) as the strongest tabular model. The notebook reports accuracy, precision, recall, F1, ROC-AUC, and PR-AUC (`average_precision`), the last being the more informative summary under class imbalance. Results are in Table 1; confusion matrices are in Figure 7.

Table 1

Classical model performance on the shared test set (phishing = positive class).

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.9995	0.9995	0.9993	0.9994	1.0	1.0
Random Forest	0.9999	0.9999	0.9998	0.9999	1.0	1.0
XGBoost	0.9999	1.0000	0.9998	0.9999	1.0	1.0

Class-Imbalance Handling

The imbalance here is mild (about 57/43), so the notebook starts with the cheap option, `scale_pos_weight`, and treats SMOTE as an explicit ablation rather than a default. Three setups are compared on the same XGBoost model: no balancing, class weighting, and SMOTE oversampling (Table 2). The three rows are nearly identical: on mild imbalance with an already-separable feature set, resampling buys essentially nothing (and SMOTE synthesizes points among correlated tabular features, a known overfitting risk). The notebook keeps `scale_pos_weight` as the cheap default.

Tuning the Strongest Model

`RandomizedSearchCV` over XGBoost is scored on PR-AUC (`average_precision`) and cross-validated with `StratifiedGroupKFold` so domains never straddle a fold boundary during tuning either.

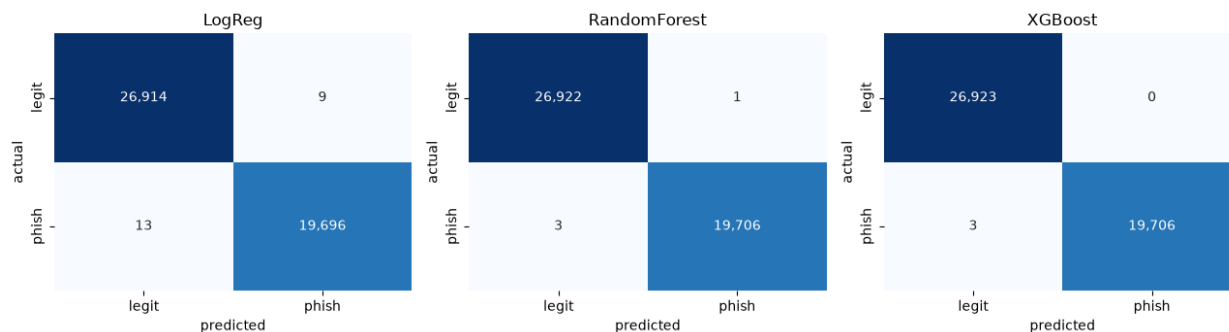


Figure 7
Confusion matrices for the three classifiers. Rows are the true class, columns the prediction, with phishing as the class of interest.

Feature Importance

The notebook uses two views: XGBoost’s built-in gain importance (Figure 8) and model-agnostic permutation importance (the drop in PR-AUC when a feature is shuffled), computed on a test-set sample. It specifically checks where the four engineered features land.

Does a Handful of Features Match the Full Model?

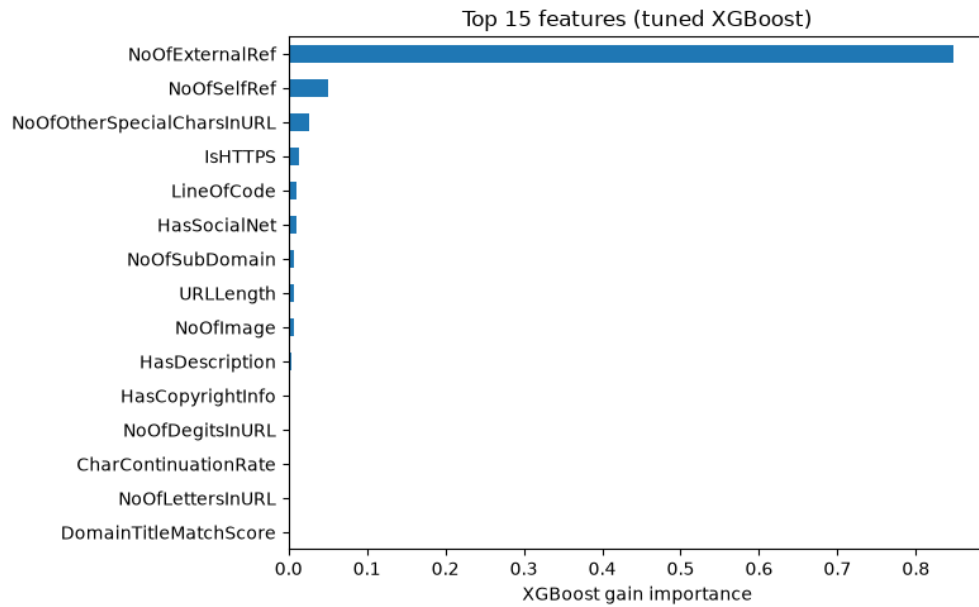
The assignment calls out this pattern as the model of a good result: if a model using only the top-K features nearly matches the full model, that is an actionable finding, because a production filter could collect far fewer signals per URL. Retraining XGBoost on just the top eight features (`NoOfExternalRef`, `NoOfSelfRef`, `NoOfOtherSpecialCharsInURL`, `IsHTTPS`, `LineOfCode`, `HasSocialNet`, `NoOfSubDomain`, `URLLength`) nearly matches the tuned full model (Table 3).

Cost-Sensitive Decision Layer

Accuracy near the ceiling hides the real design question: where do you want the few remaining errors to fall? That depends on the cost of each error type, which is asymmetric for phishing. A false negative (phishing let through) risks credential theft or fraud, so its cost is set an order of magnitude higher, anchored loosely to public breach-cost figures. A false positive (legitimate site blocked) costs user friction and support load, an illustrative

Table 2*Imbalance-handling ablation on XGBoost.*

Setup	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
none	0.9999	1.0	0.9998	0.9999	1.0	1.0
scale_pos_weight	0.9999	1.0	0.9998	0.9999	1.0	1.0
SMOTE	1.0000	1.0	0.9999	0.9999	1.0	1.0

**Figure 8**

Top-15 XGBoost gain importances. Page-content and URL-structure columns dominate; the four engineered features do not appear in the top ranks.

Table 3*Full tuned model versus a top-8-feature model.*

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
XGBoost (tuned)	0.9999	1.000	0.9998	0.9999	1.0	1.0
XGBoost (top 8)	0.9994	0.999	0.9995	0.9993	1.0	1.0

estimate rather than a sourced figure. Sweeping the decision threshold and picking the operating point that minimizes expected cost per 1,000 URLs (Figure 9) yields a minimum-cost threshold of 0.24, at \$21.44 per 1,000 URLs, versus \$32.17 at the default threshold of 0.5.

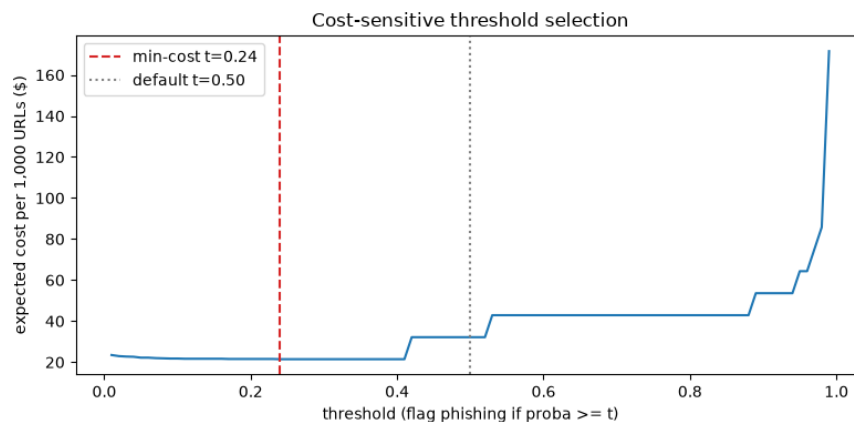


Figure 9

Expected cost per 1,000 URLs as a function of decision threshold. The minimum sits at 0.24, below the default 0.5, reflecting the higher cost assigned to false negatives.

Business Metric: It Depends on the Deployment

There is no single “right” metric; it follows the deployment scenario. A browser warning system (Google Safe Browsing style) is precision-first, because wrongly blocking a legitimate site is highly visible, so it accepts lower recall for very high precision. A mail or endpoint gateway that quarantines silently is recall-first, because a missed phishing email reaching an inbox is the expensive outcome. The notebook reports both operating points plus a three-tier allow/warn/block policy, which is closer to how real filters behave than one binary cutoff: a precision-first point at threshold 0.000 (precision 0.9900, recall 1.0000), a recall-first point at threshold 0.412 (precision 1.0000, recall 0.9999), and a three-tier policy that, on the test set, allows 26,926 URLs, blocks 19,704, and warns on 2.

Summary and Hand-Off

- **The engineered features barely move the needle here:** they land at the bottom of the importance ranking (ranks 22–54, about zero gain) and their by-class

medians overlap heavily. That is itself an honest result. The shipped page-content columns (`IsHTTPS`, `LineOfCode`, `NoOfExternalRef`, and so on) already carry essentially all the separating signal, so lexical entropy and brand-distance add little *on top of them*. They may still earn their keep in the URL-only regime (Notebook 03), where those content columns do not exist.

- **Mild imbalance needs only class weighting**; SMOTE added nothing here and risks overfitting correlated tabular features.
- **The problem is near-saturated on tabular features** (about 99.9%, matching Notebook 01 and the published PhiUSIIL numbers), so the interesting work is what to do with the tiny error budget: the cost-sensitive layer and the precision-first versus recall-first operating points make that an explicit, business-driven choice rather than a default 0.5.
- **A top-8-feature model nearly matches the full model**, an actionable result for a cheaper production filter.

The harder, more honest difficulty regime, a model that never sees the engineered or content features, is Notebook 03’s character-level model on the raw URL string, alongside the clustering and the full state-of-the-art comparison.

Character-Level Model, Clustering, and State of the Art

This section reproduces Notebook 03 (author: Pranav Dhinakar). Following Notebook 02, phishing is modeled as the positive class so precision, recall, and PR-AUC describe catching phishing. Notebook 01 established the leakage-clean feature set and split conventions, and Notebook 02 built the tabular models and saved their metrics. This notebook (1) settles the train/test grouping convention and locks it to Notebook 02’s for comparability, (2) trains a character-level CNN on the raw URL string only as the leakage-immune “hard mode” comparison, (3) clusters the phishing rows into attack “families,” and (4) assembles the state-of-the-art comparison table.

Registered-Domain Grouping Decision

Notebook 01 split on the exact `Domain` string and flagged that this *under*-groups subdomains: `ipfs.io`, `gateway.ipfs.io` and `cf-ipfs.com` are the same service but count as three domains, so related rows can still land on both sides of a train/test split. The stricter alternative groups by registered domain (eTLD+1 via `tldextract`), which collapses those into one group. This matters because the character model, the clustering, and Notebook 02’s tabular models must all use the *same* split, or the SOTA table compares numbers produced under different conditions. Three grouping strictnesses, run through the same quick logistic model, give the results in Table 4.

Table 4

Effect of grouping strictness on a quick logistic-regression score.

Grouping	Groups	Accuracy	F1
exact <code>Domain</code> (NB 01 / NB 02)	220,086	0.9995	0.9996
PaaS-aware eTLD+1 (PSL-private on)	197,709	0.9993	0.9994
naive eTLD+1	175,509	0.9990	0.9992

Accuracy is essentially invariant (0.9990–0.9995) across a 44,577-group range of strictness. The drop is monotonic with strictness, the direction leakage would predict, but negligible in size, reinforcing Notebook 01’s finding that the dataset is easy regardless of how conservatively the split is drawn. Because the score is grouping-invariant, the notebook **adopts Notebook 02’s exact Domain grouping** (`GroupShuffleSplit`, `test_size=0.2`, `random_state=42`) rather than reworking it. This makes the character model share Notebook 02’s exact test set, so the SOTA table compares numbers produced under identical conditions. The stricter registered-domain analysis is retained as evidence that this choice does not inflate the results.

Character-Level Model on the Raw URL

The tabular models in Notebook 02 use page-content and lexical features that make PhiUSIIL trivially easy (about 99.9%). This model sees only the raw URL string: no

engineered features, no page content, no `URLSimilarityIndex`. It cannot leak, because it has access to nothing but the characters of the URL itself, so it is the leakage-immune comparison point for the URL-only lexical literature (Sahingoz et al. (2019) reached 97.98% with an NLP-feature random forest). Each URL is tokenized at the character level, embedded, and passed through a small 1-D CNN with a sigmoid output, trained with class weights on the same exact-`Domain` split used in Notebook 02 so the numbers are comparable.

Architecture

The model is a compact character-level CNN (Kim (2014), adapted to characters). An embedding maps each character id to a 48-dimensional vector (`padding_idx=0`). Three parallel 1-D convolutions of kernel widths 3, 5, and 7 (128 filters each) read character 3-, 5-, and 7-grams. A global max-pool over position is followed by concatenation of the three widths, dropout (0.4), a linear layer, and a single logit. The loss is `BCEWithLogitsLoss` with `pos_weight = n_neg/n_pos` for class weighting. Inputs use `MAX_LEN = 200`, which covers roughly the 99th percentile of URL lengths, with longer URLs truncated. A grouped 10% validation slice (domains do not straddle train/val) is used only to monitor convergence and keep the best-PR-AUC epoch; the test set is untouched until evaluation.

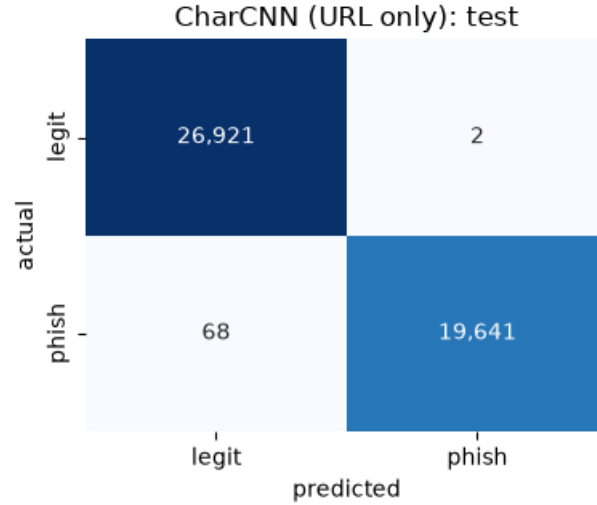
Findings

The model converged in one epoch (validation PR-AUC 0.9988 at epoch 1). On the held-out test set its metrics essentially tie the tuned tabular model (Table 5). At threshold 0.5 it makes 1 false positive and 69 false negatives on 46,632 test URLs (Figure 10).

Table 5

CharCNN (URL only) versus tuned XGBoost (Notebook 02).

Metric	CharCNN (URL only)	tuned XGBoost (NB 02)
accuracy	0.9985	~0.999
precision (phishing)	0.9999	~0.999
recall (phishing)	0.9965	~0.999
PR-AUC	0.9993	~0.999

**Figure 10**

CharCNN confusion matrix on the 46,632-row test set: 1 false positive and 69 false negatives at threshold 0.5.

The URL-only model does not come in harder than the tabular models. It essentially ties them and lands above the URL-only lexical literature (Sahingoz et al. (2019): 97.98%; the char-CNN band in the SOTA comparison: 97–99%). So PhiUSIIL’s separability is not confined to the page-content columns Notebook 01 flagged; it is present in the raw URL strings themselves. Legitimate and phishing URLs here come from systematically different sources (well-known registrable domains versus IP-address hosts, IPFS gateways, PaaS subdomains, randomized tokens), differences visible character by character. This is the project’s own evidence for the benchmark-leakage critique in the SOTA comparison (Dalton et al., 2025; Das et al., 2019).

Two caveats apply to the SOTA table. First, this model beats the URL-only literature because PhiUSIIL is more separable than the cross-source corpora those papers used, not because the architecture is stronger; the claim is “about 99.85% on PhiUSIIL,” not a general result. Second, the exact-`Domain` split still lets registered-domain siblings on shared platforms (`web.app`, `firebaseapp.com`, `repl.co`) fall on both sides, which a character model could exploit as platform-level memorization. The grouping analysis showed the tabular score is near-invariant to this (0.9993 PaaS-aware versus 0.9995 exact),

so it is unlikely to be the main driver, but the CNN is more exposed to it than logistic regression.

Clustering: Phishing Attack Families

The models so far are supervised. This section adds an unsupervised method, the project’s second algorithm type, and asks a different question: not whether a URL is phishing, but what kinds of phishing are in the data. The notebook clusters the phishing rows only on Notebook 01’s leakage-clean feature set (label held out) and profiles the groups as attack “families.” This is standard in the phishing literature (Feng et al., 2022 clustered phishing kits; Althobaiti et al., 2023 clustered campaigns; Zia and Kalidass, 2025 clustered lexical URL tokens). Cluster IDs are not fed back into the classifier, to avoid a second leakage-review cycle. The method is KMeans with k chosen by elbow and silhouette, centroid profiling to name the families, PCA for a 2-D view, and HDBSCAN as a density-based cross-check.

Choosing k

The elbow and silhouette scans over k appear in Figure 11 and Table 6. Here $k = 2$ has by far the highest silhouette (0.44): the strongest structure is a single binary split, matching Notebook 01’s finding that phishing pages divide into content-less shells versus content-bearing pages. The elbow is otherwise soft, typical of high-dimensional tabular data with correlated features. Because $k = 2$ is too coarse for a families deliverable, the notebook uses $k = 5$, the silhouette peak among $k \geq 3$ and where the elbow flattens. It reports the $k = 2$ split as the dominant coarse structure and profiles $k = 5$ as interpretable strata. The silhouettes at $k = 5$ (about 0.15) are low, so these are overlapping descriptive families rather than crisply separated clusters; HDBSCAN checks how much crisp density structure exists.

The Five Families

Each cluster’s raw-unit means and its centroid’s top distinguishing features (in standard deviations from the phishing-wide mean) give the families in Table 7.

Table 6*KMeans model-selection scan over k (silhouette on a 10k sample).*

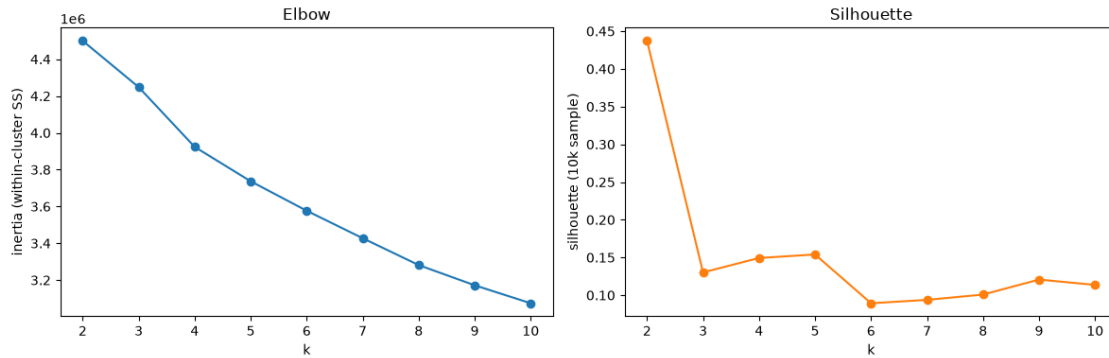
k	Inertia	Silhouette (10k)
2	4,500,166	0.437
3	4,248,341	0.130
4	3,923,595	0.149
5	3,736,818	0.154
6–10	3.58M–3.07M	0.089–0.121

Empty-shell pages dominate at 68%, consistent with Notebook 01’s finding that most phishing rows are content-less and with the $k = 2$ super-split (87/13) that separates these from content-bearing pages. The two active attack types separate cleanly: credential-harvesting forms (c3), with the password/submit/hidden-field signature of a fake login, and HTTP lookalike stubs (c4), which look brand-consistent but lack HTTPS. Cluster c1 holds only 5 rows; KMeans minimizes squared distance and is sensitive to outliers, so a few 4,000-character URLs captured their own centroid. This is a KMeans artifact, not a real family, and the HDBSCAN cross-check relabels these as noise.

What the Clustering Shows

KMeans ($k = 5$) partitions phishing into interpretable strata: one dominant empty-shell family (68%) and two distinct active types, credential-harvesting forms (12%, password/submit/hidden-field signature) and HTTP lookalike stubs (19%, domain-matching titles but no HTTPS), plus a small IP-host / obfuscated-URL group (1%) and a 5-row outlier artifact. This mirrors the supervised EDA in Notebook 01: most phishing pages are content-less shells, and the rest split into “steal credentials directly” versus “look legitimate cheaply” (Figure 12).

The structure is soft. Three signals say these are convenient strata over a partly continuous space rather than sharply separated kinds. First, silhouettes are low (about 0.15 at $k = 5$; only the $k = 2$ empty-versus-content split scores well, at 0.44). Second, PCA is diffuse: the first two components explain 22.9% of variance, and the projection is

**Figure 11**

Elbow (inertia) and silhouette curves for KMeans. The silhouette peaks sharply at $k = 2$; $k = 5$ is the local peak used for interpretable families.

one dense overlapping mass with a few outlier spikes (the 5 long-URL points, one IP-host page, the credential-form tail), not five islands. Third, HDBSCAN found 25 micro-clusters and left 30% as noise on a 25k sample, rather than five clean families. The micro-clusters likely correspond to near-identical phishing kits (templated pages reused across many URLs), the finer level Feng et al. (2022) found; the 30% is genuinely continuous. The 5-row KMeans outlier does not survive as an HDBSCAN cluster, confirming it was an artifact. So KMeans $k = 5$ is the descriptive lens that names recognizable attack families and ties them to the supervised findings, while HDBSCAN is the validity check showing the true structure is a few dominant modes plus many small kit-level pockets and a continuous remainder. The contrast between partition-based and density-based clustering is itself a result for the write-up.

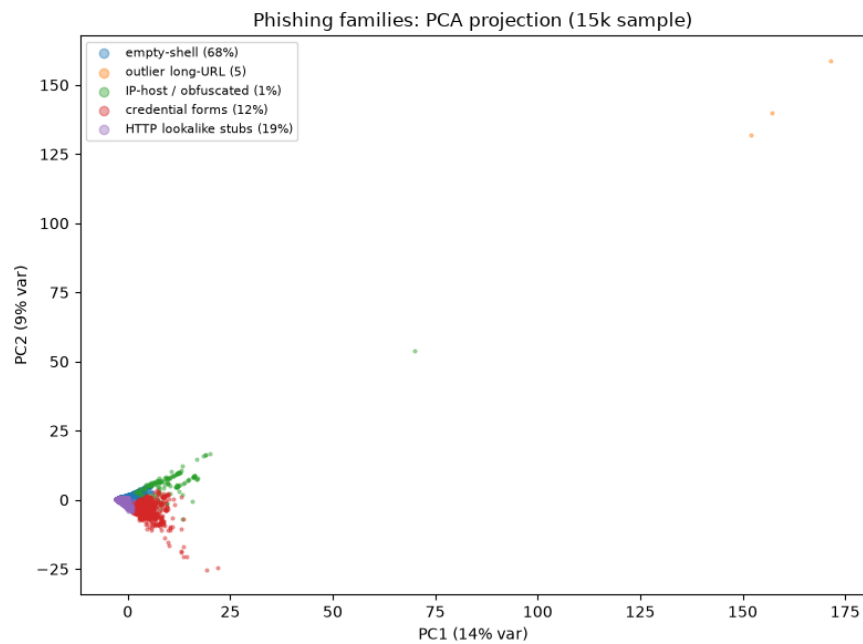
State-of-the-Art Comparison

Two framing rules govern the comparison. First, numbers are comparable only within a dataset; accuracy on PhiUSIIL and accuracy on another corpus are different measurements, so the CharCNN’s about 99.85% is on PhiUSIIL, directly comparable to other PhiUSIIL numbers and only contextually comparable to URL-only models on other corpora. Second, published numbers are flagged by verification status: several come from secondary citations because the primary papers were not directly accessible, and sources

Table 7

The five phishing families from KMeans ($k = 5$).

Cluster (size)	Attack family	Defining signature
c0 – 68,743 (68.4%)	Empty-shell pages	No title, no forms, about 40 lines of code, low title-match (parked/stub, no real content)
c3 – 11,809 (11.7%)	Credential-harvesting forms	Password, submit, hidden fields, images, CSS, favicon, about 267 lines of code (fake logins)
c4 – 18,830 (18.7%)	HTTP lookalike stubs	Title matches domain, short clean URLs, only 17% HTTPS (brand-consistent, plain HTTP)
c2 – 1,133 (1.1%)	IP-host / obfuscated pages	HasObfuscation , 39% IP-hosted, long URLs, many ? params
c1 – 5 (0.0%)	Outlier micro-cluster	URL length about 4,128 chars, extreme obfuscated/digit/ampersand counts (ultra-long parameter-stuffed URLs)

**Figure 12**

PCA projection of the phishing families (15k sample). The projection is one dense overlapping mass with a few outlier spikes rather than five separated islands; the first two components explain 22.9% of variance.

sometimes disagree, so those are marked unverified and treated as directional. Figure 13 places the project’s supervised models against the published PhiUSIIL band.

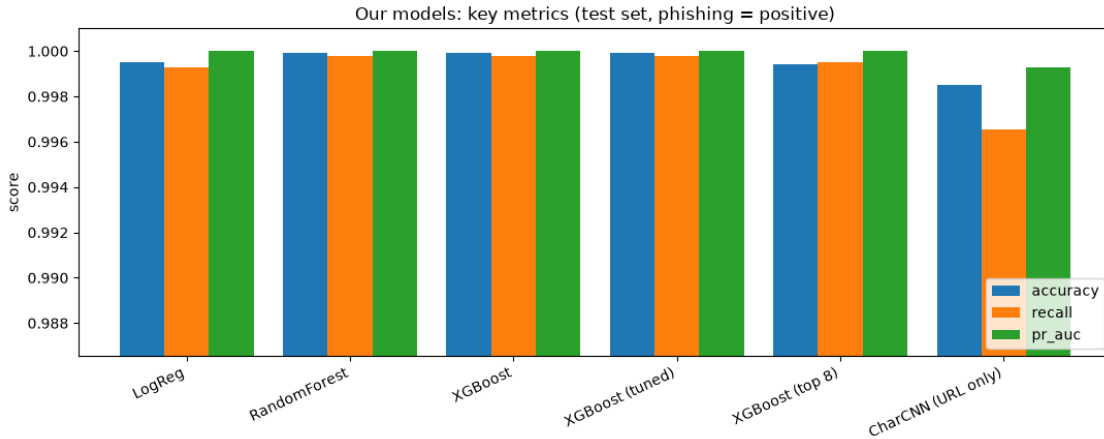


Figure 13

The project’s models against the published PhiUSIIL 99.5–100% band. All five supervised models fall inside the band; the CharCNN is the informative near-tie.

Published Results in Context

Table 8 lists results on the same dataset (PhiUSIIL), directly comparable to the project’s rows, and Table 9 lists URL-only and lexical results on other datasets, included for context only.

These bracket a 97–99% range for URL-only models on harder cross-source corpora. The CharCNN lands above that band because PhiUSIIL is more separable at the raw-string level, not because the architecture is stronger. Precedents for the other methods: for the imbalance ablation, Omari et al. (2025) report SMOTE-NC + XGBoost at 98.0% precision / 98.5% recall; for the clustering, Feng et al. (2022) (k-medoid on phishing kits, 90.1% TPR) and Zia and Kalidass (2025) (LSH on lexical tokens, 93% on PhishStorm) frame attack-family clustering as standard practice.

Where Our Results Land

Against PhiUSIIL, every one of the project’s models sits in the published 99.5–100% band: logistic regression 0.9995, random forest 0.9999, XGBoost 1.0000, tuned XGBoost 0.9999, and a top-8-feature model at 0.9993. This reproduces the dataset authors’ range

Table 8*Published results on PhiUSIIL, with the project’s own rows.*

Study	Approach	Reported / status
Prasad and Chandra (2024) (dataset authors)	similarity-index + incremental learning	~99.2–99.8% (one source 99.97%); unverified, sources disagree
Pentapalli et al. (2025)	TSK fuzzy; excluded <code>URLSimilarityIndex</code>	qualitative agreement with our leakage call
Independent reproductions (Kaggle, small papers)	RF / DT / KNN / LR / XGB / FCNN / BERT	99.5–100% band; aggregated, unverified
This project, tuned XGBoost (NB 02)	full leakage-clean feature set	see Table 3
This project, CharCNN (URL only)	raw URL characters only	0.9985 acc / 0.9993 PR-AUC

and the independent reproductions across model families; it confirms a known ceiling rather than producing an outlier. At four decimals almost everything rounds to about 1.0, so accuracy ordering is noise. The informative comparison is the CharCNN: 0.9985 accuracy and 0.9993 PR-AUC on the same 46,632-row test split, within 0.0015 accuracy of the full-feature XGBoost. A model denied `URLSimilarityIndex`, the page-content flags, and the engineered features, seeing only the raw URL string, matches models that see all 49 clean features. That is direct evidence from the project’s own pipeline that PhiUSIIL’s separability is in the URL strings, not just the content columns Notebook 01 flagged, and it corroborates the benchmark-leakage critique rather than only citing it. The supervised problem is saturated; the useful question is why. The contribution of this notebook is not a new ceiling number but the leakage-immune CharCNN and the clustering, which together characterize what makes the benchmark easy and what kinds of phishing it contains.

Synthesis and Discussion

The three notebooks converge on one claim: on PhiUSIIL, the separability of phishing from legitimate URLs is intrinsic to the data, not an artifact of a single leaky column or a favorable split. The project demonstrates this three separate ways. The data audit showed that removing the obvious leak, `URLSimilarityIndex`, leaves the remaining

Table 9*URL-only / lexical literature on other datasets (context only).*

Study	Approach	Reported acc	Status
Sahingoz et al. (2019)	RF on 39 NLP lexical features	97.98%	corroborated; firmest external number
Yerima and Alzaylaee (2020)	char-CNN	98.2% (F1 0.976)	reported (arXiv)
Maneriker et al. (2021) (URLTran)	BERT on subword URLs	about 96%	reported (arXiv)
MiniLM-CNN-LSTM authors (2024)	MiniLM-CNN-LSTM hybrid	98.98%	reported
Dubey et al. (2025)	char-CNN + LightGBM	99.82%	reported (arXiv)
Mohammad et al. (2014)	self-structuring NN	92–96% (disputed)	unverified; hedge or drop

features at about 99.9%. The grouping analysis showed that tightening the train/test split from exact-domain to registered-domain grouping, across a range of more than 44,000 groups, moves accuracy only from 0.9995 to 0.9990. And the character model, which sees nothing but the raw URL string and therefore cannot leak, still reached 0.9985 accuracy. Three different attempts to make the problem harder all failed to move the number in any meaningful way. That consistency is the result.

The project also satisfies the course requirement to combine at least two algorithm families under experimental comparison. It uses supervised classification (logistic regression, random forest, and XGBoost) and unsupervised clustering (KMeans and HDBSCAN), and it runs several controlled comparisons within those: an imbalance ablation (none versus class weighting versus SMOTE), hyperparameter tuning with grouped cross-validation, a full-model versus top-8-feature comparison, a partition-based versus density-based clustering cross-check, and a character-level model set against both the tabular models and the published literature. The empirical comparisons are presented graphically where possible, as the assignment prefers.

Two things are worth stating about what the project does and does not claim. It does claim that PhiUSIIL is easy for reasons that go all the way down to the raw URL characters, and that the easy-benchmark result should be read as evidence about the

dataset rather than about any one model. It does not claim a general phishing-detection breakthrough. The character model beats the URL-only literature because PhiUSIIL is more separable than the cross-source corpora those studies used, not because the architecture is stronger. The honest phrasing is “about 99.85% on PhiUSIIL,” full stop.

A consistent caveat runs through the state-of-the-art comparison: many of the published numbers are vendor-reported or come from secondary citations that were not independently verified, and sources sometimes disagree. Those numbers are treated as directional rather than exact, and the tables flag their verification status. The safest reading of the comparison is that the project’s results reproduce a known ceiling, not that any single external figure is authoritative.

The main limitations follow from the benchmark itself. Because PhiUSIIL is so separable, the accuracy ordering among models is noise, so the useful work shifted to interpretation: the cost-sensitive operating points, the feature-importance finding that a handful of features suffices, and the clustering that names attack families. The exact-domain split also leaves platform-level siblings on shared hosting on both sides of the split, which the character model is more exposed to than the linear model; the grouping analysis suggests this is not the main driver, but a fully platform-aware split would close the gap. Natural next steps are to test the trained models on a different corpus to measure how much of the performance is PhiUSIIL-specific, and to feed the kit-level structure that HDBSCAN surfaced into a more operational grouping of phishing campaigns.

Conclusion

This project built and compared a full phishing-URL detection pipeline on PhiUSIIL: a leakage audit and exploratory analysis, three classical classifiers with imbalance, tuning, feature-importance and cost-sensitive analyses, a character-level convolutional model on the raw URL string, and an unsupervised clustering of phishing into attack families. Every supervised model landed in the published 99.5–100% band, and the leakage-immune character model essentially tied them. The contribution is not a new

accuracy number but an explanation of why the benchmark is easy, established from three independent directions, together with a characterization of the kinds of phishing the dataset contains. The result is a report that treats near-perfect accuracy as something to be explained rather than celebrated.

References

- Althobaiti et al. (2023). *Clustering phishing campaigns* [VERIFY full citation (authors, venue).].
- Dalton et al. (2025). *On benchmark leakage in phishing detection* [VERIFY full citation (authors, venue).].
- Das, A., Baki, S., El Aassal, A., Verma, R., & Dunbar, A. (2019). SoK: A comprehensive reexamination of phishing research from the security perspective [VERIFY volume/pages/DOI.]. *IEEE Communications Surveys & Tutorials*.
- Dubey et al. (2025). *Character-CNN with LightGBM for phishing detection* [VERIFY full author list and title.]. arXiv: 2512.16717.
- Feng et al. (2022). Clustering phishing kits [VERIFY author list and title.]. *PeerJ Computer Science*, 8, e868.
- Kim, Y. (2014). Convolutional neural networks for sentence classification [arXiv:1408.5882]. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1746–1751.
- Maneriker, P., et al. (2021). *URLTran: Improving phishing URL detection using transformers* [VERIFY full author list.]. arXiv: 2106.05256.
- MiniLM-CNN-LSTM authors. (2024). A MiniLM-CNN-LSTM hybrid for phishing URL detection [MDPI. VERIFY authors, volume, pages.]. *Technologies*.
- Mohammad, R. M., Thabtah, F., & McCluskey, L. (2014). Predicting phishing websites based on self-structuring neural network. *Neural Computing and Applications*, 25(2), 443–458.
- Omari et al. (2025). SMOTE-NC with XGBoost for phishing detection [VERIFY full author list, title, pages.]. *International Journal of Advanced Computer Science and Applications*, 16(2).
- Pentapalli et al. (2025). *Phishing detection using a TSK fuzzy classifier* [VERIFY full author list and title.]. arXiv: 2504.18636.

- Prasad, A., & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning [VERIFY exact page/DOI. Dataset also at UCI ML Repository, id 967.]. *Computers & Security*, 136, 103545.
- Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357.
- Yerima, S. Y., & Alzaylaee, M. K. (2020). *High accuracy phishing detection based on convolutional neural networks* [VERIFY exact title/venue.]. arXiv: 2004.03960.
- Zia & Kalidass. (2025). *Locality-sensitive hashing of lexical URL tokens* [VERIFY full author list and title.]. arXiv: 2502.13171.

Appendix A

Contribution Statement

The project was divided by notebook, with each member owning one notebook end to end and the report assembled collaboratively.

- **Guna Pasupathy** authored Notebook 01 (data loading, the three-part leakage audit, exploratory data analysis, and the saved leakage-clean feature manifest that the other notebooks consume).
- **Jason Justice** authored Notebook 02 (feature engineering from the raw URL/Domain strings, the three classical classifiers, the class-imbalance ablation, hyperparameter tuning, feature-importance analysis, the top-8-feature comparison, and the cost-sensitive decision layer).
- **Pranav Dhinakar** authored Notebook 03 (the registered-domain grouping decision, the character-level CNN on raw URLs, the KMeans/HDBSCAN clustering of attack families, and the state-of-the-art comparison) and assembled this report.

[Team: confirm the exact split of report writing, presentation, and repository maintenance here before submission.]

Appendix B

Generative-AI Use Disclosure

Consistent with the course policy, the team discloses its use of generative-AI tools. All modeling code, experimental design, analysis, and numerical results in the three notebooks are the team's own work. A large-language-model assistant was used to help assemble and format this LaTeX report from the notebooks' existing author-written markdown notes, to convert notebook tables and figures into report tables and captions, and to draft the introduction, synthesis, and conclusion sections, which were then reviewed and edited by the authors. The assistant did not generate the analysis or the results. Any AI-assisted passages were checked against the notebook outputs for accuracy. *[Team: add any other tools used during development, for example code assistance in the notebooks, and adjust this statement to match.]*

Appendix C

Source Code

The complete, documented source code for the three notebooks is listed below, exported from the committed Jupyter notebooks. The authoritative version, with full commit history and outputs, is in the team GitHub repository linked on the title page.

Notebook 01 – Load, Leakage, EDA (Guna Pasupathy)

```

1  # Notebook 01 - Load, Leakage, EDA (Guna Pasupathy)
2  # Exported from 01_load_leakage_eda.ipynb (code cells only, in order).
3
4  # ===== Code cell 1 =====
5  import json
6  from pathlib import Path
7
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11 import seaborn as sns
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.metrics import accuracy_score, f1_score
14 from sklearn.model_selection import GroupShuffleSplit, train_test_split
15 from sklearn.preprocessing import StandardScaler
16
17 DATA_PATH = Path("../data/phiusiil.csv")
18
19 # ===== Code cell 2 =====
20 df = pd.read_csv(DATA_PATH)
21 y = df["label"]
22
23 print(f"shape: {df.shape}")
24 print(f"missing values: {df.isna().sum().sum()}")
25 print()
26 print("column types:")
27 print(df.dtypes.value_counts())
28 df.head(3)
29
30 # ===== Code cell 3 =====
31 counts = y.value_counts().sort_index()
32
33 plt.figure(figsize=(5, 4))
34 plt.bar(
35     ["phishing (0)", "legitimate (1)"],
36     counts.values,
37     color=["tab:orange", "tab:blue"],
38 )
39 for i, c in enumerate(counts.values):
40     plt.text(i, c + 2000, f"{c:,} ({c / len(df):.1%})", ha="center")
41 plt.ylabel("rows")
42 plt.title("Class balance")

```

```

43 plt.ylim(0, counts.max() * 1.15)
44 plt.tight_layout()
45 plt.show()
46
47 # ===== Code cell 4 =====
48 dup_url_extra = int(df["URL"].duplicated().sum())
49 dups = df[df["URL"].duplicated(keep=False)]
50 conflicts = dups.groupby("URL")["label"].nunique()
51
52 print(f"exact duplicate rows: {df.duplicated().sum()}")
53 print(f"repeated URLs (extra rows): {dup_url_extra}")
54 print(f"repeated URLs with conflicting labels: {(conflicts > 1).sum()}")
55
56 # ===== Code cell 5 =====
57 print(df.groupby("label")["URLSimilarityIndex"].describe())
58
59 n_100 = ((df["URLSimilarityIndex"] == 100) & (y == 0)).sum()
60 print(f"\nphishing rows that also score exactly 100: {n_100}")
61
62 # ===== Code cell 6 =====
63 bins = np.linspace(0, 100, 41)
64
65 plt.figure(figsize=(8, 4))
66 plt.hist(
67     df.loc[y == 1, "URLSimilarityIndex"],
68     bins=bins,
69     alpha=0.7,
70     color="tab:blue",
71     label="legitimate (1)",
72 )
73 plt.hist(
74     df.loc[y == 0, "URLSimilarityIndex"],
75     bins=bins,
76     alpha=0.7,
77     color="tab:orange",
78     label="phishing (0)",
79 )
80 plt.yscale("log")
81 plt.xlabel("URLSimilarityIndex")
82 plt.ylabel("count (log scale)")
83 plt.title("URLSimilarityIndex by class")
84 plt.legend()
85 plt.tight_layout()
86 plt.show()
87
88 # ===== Code cell 7 =====
89 def quick_logreg(X, y, groups=None):
90     """Train a basic logistic regression, return test acc and F1."""
91     if groups is None:
92         X_train, X_test, y_train, y_test = train_test_split(
93             X, y, test_size=0.2, random_state=42, stratify=y
94         )
95     else:
96         # keep all rows of the same group on one side of the split

```

```

97     gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
98     train_idx, test_idx = next(gss.split(X, y, groups=groups))
99     X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
100     y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
101
102     scaler = StandardScaler()
103     X_train = scaler.fit_transform(X_train)
104     X_test = scaler.transform(X_test)
105     model = LogisticRegression(max_iter=1000)
106     model.fit(X_train, y_train)
107     preds = model.predict(X_test)
108     return accuracy_score(y_test, preds), f1_score(y_test, preds)
109
110
111 numeric_cols = df.select_dtypes("number").columns
112 safe_cols = [
113     c for c in numeric_cols if c not in ("URLSimilarityIndex", "label")
114 ]
115
116 acc1, f11 = quick_logreg(df[["URLSimilarityIndex"]], y)
117 acc2, f12 = quick_logreg(df[safe_cols], y)
118
119 print(f"URLSimilarityIndex alone: acc={acc1:.4f} f1={f11:.4f}")
120 print(f"all other numeric cols: acc={acc2:.4f} f1={f12:.4f}")
121
122 # ===== Code cell 8 =====
123 binary_cols = [
124     c for c in numeric_cols if c != "label" and set(df[c].unique()) <= {0, 1}
125 ]
126
127 rates = df.groupby("label")[binary_cols].mean().T
128 rates.columns = ["phishing_rate", "legit_rate"]
129 rates["gap"] = (rates["legit_rate"] - rates["phishing_rate"]).abs()
130 rates = rates.sort_values("gap", ascending=False)
131
132 print(f"{len(binary_cols)} binary feature columns")
133 rates.head(10).round(3)
134
135 # ===== Code cell 9 =====
136 giveaways = rates.index[:6].tolist()
137 print("dropping:", giveaways)
138
139 acc3, f13 = quick_logreg(df[[c for c in safe_cols if c not in giveaways]], y)
140 print(f"without giveaway flags: acc={acc3:.4f} f1={f13:.4f}")
141
142 # ===== Code cell 10 =====
143 domain_counts = df["Domain"].value_counts()
144 repeated = domain_counts[domain_counts > 1]
145
146 print(f"unique domains: {len(domain_counts):,} / {len(df):,} rows")
147 print(
148     f"domains appearing more than once: {len(repeated):,} "
149     f"({repeated.sum():,} rows, {repeated.sum() / len(df):.1%})"
150 )

```

```

151 domain_counts.head(10)
152
153 # ===== Code cell 11 =====
154 acc4, f14 = quick_logreg(df[safe_cols], y, groups=df["Domain"])
155
156 print(f"normal random split: acc={acc2:.4f} f1={f12:.4f}")
157 print(f"domain-grouped split: acc={acc4:.4f} f1={f14:.4f}")
158
159 # ===== Code cell 12 =====
160 print(f"unique TLDs: {df['TLD'].nunique()}")
161 print(df["TLD"].value_counts().head(10))
162
163 weird = df["TLD"].astype(str).str.fullmatch(r"\d+")
164 is_ip = df.loc[weird, "IsDomainIP"] == 1
165 print(f"\nrows where the 'TLD' is just a number: {weird.sum()}")
166 print(f"of those, IsDomainIP == 1: {is_ip.sum()}")
167 print(f"labels of those rows: {df.loc[weird, 'label'].unique()}")
168 df.loc[weird, ["Domain", "TLD", "label"]].head(3)
169
170 # ===== Code cell 13 =====
171 legit_share_per_tld = df.groupby("TLD")["label"].mean()
172 legit_share = df["TLD"].map(legit_share_per_tld)
173
174 corr = legit_share.corr(df["TLDLegitimateProb"])
175 print(f"corr with per-TLD legit share in this data: {corr:.3f}")
176
177 # ===== Code cell 14 =====
178 corrs = df[safe_cols].corrwith(y)
179 top15 = corrs.reindex(corrs.abs().sort_values().index).tail(15)
180 colors = ["tab:blue" if v > 0 else "tab:red" for v in top15]
181
182 plt.figure(figsize=(8, 6))
183 plt.barh(top15.index, top15.values, color=colors)
184 plt.axvline(0, color="gray", linewidth=1)
185 plt.xlabel("correlation with label (positive = legitimate)")
186 plt.title("Top 15 safe features by correlation with label")
187 plt.tight_layout()
188 plt.show()
189
190 # ===== Code cell 15 =====
191 non_binary = [c for c in safe_cols if c not in binary_cols]
192
193 medians = df.groupby("label")[non_binary].median().T
194 medians.columns = ["phishing_median", "legit_median"]
195 print(f"{len(non_binary)} non-binary numeric features")
196 medians.round(3)
197
198 # ===== Code cell 16 =====
199 feats = [
200     "URLLength",
201     "DomainLength",
202     "DegitRatioInURL",
203     "SpacialCharRatioInURL",
204     "LineOfCode",

```

```

205     "NoOfJS",
206     "NoOfImage",
207     "NoOfExternalRef",
208 ]
209
210 fig, axes = plt.subplots(2, 4, figsize=(14, 6))
211 for ax, col in zip(axes.flat, feats):
212     # clip extreme outliers so the plots stay readable
213     clipped = df[col].clip(upper=df[col].quantile(0.99))
214     ax.hist(
215         clipped[y == 1],
216         bins=30,
217         alpha=0.6,
218         color="tab:blue",
219         density=True,
220         label="legitimate",
221     )
222     ax.hist(
223         clipped[y == 0],
224         bins=30,
225         alpha=0.6,
226         color="tab:orange",
227         density=True,
228         label="phishing",
229     )
230     ax.set_title(col, fontsize=10)
231     ax.set_yticks([])
232 axes[0, 0].legend()
233 fig.suptitle("Feature distributions by class (clipped at 99th pct)")
234 plt.tight_layout()
235 plt.show()
236
237 # ===== Code cell 17 =====
238 corr_matrix = df[safe_cols].corr()
239 upper = corr_matrix.where(np.triu(np.ones_like(corr_matrix, dtype=bool), k=1))
240 pairs = upper.stack()
241 high = pairs[pairs.abs() > 0.8].sort_values(ascending=False)
242
243 print("feature pairs with |corr| > 0.8:")
244 print(high.round(3))
245
246 # ===== Code cell 18 =====
247 top12 = corrs.abs().sort_values(ascending=False).head(12).index
248
249 plt.figure(figsize=(9, 7))
250 sns.heatmap(
251     df[list(top12)].corr(),
252     annot=True,
253     fmt=".2f",
254     cmap="coolwarm",
255     vmin=-1,
256     vmax=1,
257     center=0,
258 )

```

```

259 plt.title("Correlations among the 12 strongest safe features")
260 plt.tight_layout()
261 plt.show()
262
263 # ===== Code cell 19 =====
264 LEAKY_COLS = ["URLSimilarityIndex", "URL", "Domain", "Title"]
265
266 clean_features = [
267     c for c in df.columns if c not in LEAKY_COLS + ["label", "TLD"]
268 ]
269
270 manifest = {
271     "target": "label",
272     "dropped_leaky": LEAKY_COLS,
273     "dropped_identifier": [],
274     "categorical_raw": ["TLD"],
275     "drop_duplicate_urls": True,
276     "split_group_column": "Domain",
277     "clean_numeric_features": clean_features,
278 }
279
280 out_path = Path("../data/01_feature_columns.json")
281 out_path.write_text(json.dumps(manifest, indent=2))
282 print(f"saved {len(clean_features)} feature names to {out_path}")

```

Notebook 02 – Classical Models (Jason Justice)

```

1 # Notebook 02 - Classical Models (Jason Justice)
2 # Exported from 02_classical_models.ipynb (code cells only, in order).
3
4 # ===== Code cell 1 =====
5 import json
6 from pathlib import Path
7
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11 import seaborn as sns
12 from imblearn.over_sampling import SMOTE
13 from imblearn.pipeline import Pipeline as ImbPipeline
14 from rapidfuzz import process
15 from rapidfuzz.distance import Levenshtein
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.inspection import permutation_importance
18 from sklearn.linear_model import LogisticRegression
19 from sklearn.metrics import (
20     accuracy_score,
21     average_precision_score,
22     confusion_matrix,
23     f1_score,
24     precision_recall_curve,
25     precision_score,
26     recall_score,
27     roc_auc_score,

```

```

28 )
29 from sklearn.model_selection import (
30     GroupShuffleSplit,
31     RandomizedSearchCV,
32     StratifiedGroupKFold,
33 )
34 from sklearn.pipeline import Pipeline
35 from sklearn.preprocessing import StandardScaler
36 from xgboost import XGBClassifier
37
38 RANDOM_STATE = 42
39
40 # ===== Code cell 2 =====
41 DATA_DIR = Path("../data")
42 df = pd.read_csv(DATA_DIR / "phiusiil.csv")
43 manifest = json.loads((DATA_DIR / "01_feature_columns.json").read_text())
44
45 clean_features = manifest["clean_numeric_features"]
46 print(f"rows: {len(df):,}")
47 print(f"clean features from notebook 01: {len(clean_features)}")
48
49 # ===== Code cell 3 =====
50 def shannon_entropy(s):
51     if not s:
52         return 0.0
53     counts = np.unique(list(s), return_counts=True)[1]
54     probs = counts / counts.sum()
55     return float(-(probs * np.log2(probs)).sum())
56
57
58 SUFFIX_STOP = {
59     "www",
60     "com",
61     "org",
62     "net",
63     "edu",
64     "gov",
65     "co",
66     "uk",
67     "io",
68     "de",
69     "ru",
70     "info",
71     "biz",
72     "app",
73     "xyz",
74     "online",
75     "site",
76     "web",
77 }
78
79 SHORTENERS = {
80     "bit.ly",
81     "tinyurl.com",
82     "goo.gl",

```

```
82     "t.co",
83     "ow.ly",
84     "is.gd",
85     "buff.ly",
86     "adf.ly",
87     "rebrand.ly",
88     "cutt.ly",
89     "shorturl.at",
90 }
91 BRANDS = [
92     "paypal",
93     "apple",
94     "microsoft",
95     "google",
96     "amazon",
97     "facebook",
98     "instagram",
99     "netflix",
100    "linkedin",
101    "whatsapp",
102    "outlook",
103    "office365",
104    "chase",
105    "wellsfargo",
106    "bankofamerica",
107    "citibank",
108    "hsbc",
109    "barclays",
110    "americanexpress",
111    "capitalone",
112    "santander",
113    "visa",
114    "mastercard",
115    "coinbase",
116    "binance",
117    "blockchain",
118    "metamask",
119    "dropbox",
120    "adobe",
121    "yahoo",
122    "gmail",
123    "icloud",
124    "spotify",
125    "steam",
126    "roblox",
127    "twitter",
128    "tiktok",
129    "snapchat",
130    "ebay",
131    "walmart",
132    "target",
133    "fedex",
134    "ups",
135    "dhl",
```

```

136     "usps",
137     "irs",
138     "hmrc",
139     "docusign",
140     "onedrive",
141     "zoom",
142     "twitch",
143     "discord",
144     "telegram",
145     "reddit",
146     "github",
147     "wordpress",
148     "shopify",
149     "stripe",
150     "venmo",
151     "zelle",
152     "cashapp",
153     "revolut",
154     "aliexpress",
155     "alibaba",
156     "samsung",
157     "xfinity",
158     "verizon",
159     "tmobile",
160     "vodafone",
161     "booking",
162     "airbnb",
163     "uber",
164     "lyft",
165     "doordash",
166     "instacart",
167 ]
168
169
170 def domain_labels(domain):
171     return [
172         p
173         for p in domain.lower().split(".")
174         if p not in SUFFIX_STOP and len(p) >= 3
175     ]
176
177
178 def registered_domain(domain):
179     d = domain.lower()
180     return d[4:] if d.startswith("www.") else d
181
182 # ===== Code cell 4 =====
183 dom = df["Domain"].astype(str)
184
185 df["url_entropy"] = df["URL"].astype(str).map(shannon_entropy)
186 df["domain_entropy"] = dom.map(shannon_entropy)
187 df["is_punycode"] = dom.str.contains("xn--").astype(int)
188 df["is_shortener"] = dom.map(registered_domain).isin(SHORTENERS).astype(int)
189

```

```

190 # brand_distance: min normalized Levenshtein from any significant domain
191 # label to a brand. Compute once per unique label (fast in C), then map.
192 labels_per_row = dom.map(domain_labels)
193 unique_labels = sorted({lab for labs in labels_per_row for lab in labs})
194 dist = process.cdist(
195     unique_labels, BRANDS, scorer=Levenshtein.normalized_distance
196 )
197 label_min = dict(zip(unique_labels, dist.min(axis=1)))
198
199
200 def brand_distance(labs):
201     return min((label_min[x] for x in labs), default=1.0)
202
203
204 df["brand_distance"] = labels_per_row.map(brand_distance).astype(float)
205
206 engineered = [
207     "url_entropy",
208     "domain_entropy",
209     "brand_distance",
210     "is_punycod",
211     "is_shortener",
212 ]
213 print(df[engineered].describe().round(3).to_string())
214
215 # ===== Code cell 5 =====
216 phish = df["label"] == 0
217 by_class = pd.DataFrame(
218     {
219         "phishing_median": df.loc[phish, engineered].median(),
220         "legit_median": df.loc[~phish, engineered].median(),
221     }
222 )
223 print(by_class.round(3).to_string())
224
225 cont = ["url_entropy", "domain_entropy", "brand_distance"]
226 fig, axes = plt.subplots(1, 3, figsize=(13, 3.5))
227 for ax, col in zip(axes, cont):
228     ax.hist(
229         df.loc[~phish, col],
230         bins=40,
231         alpha=0.7,
232         density=True,
233         color="tab:blue",
234         label="legitimate",
235     )
236     ax.hist(
237         df.loc[phish, col],
238         bins=40,
239         alpha=0.7,
240         density=True,
241         color="tab:orange",
242         label="phishing",
243     )

```

```

244     ax.set_title(col)
245     ax.legend()
246 plt.tight_layout()
247 plt.show()
248
249 # ===== Code cell 6 =====
250 features = clean_features + engineered
251
252 before = len(df)
253 model_df = df.drop_duplicates(subset="URL").reset_index(drop=True)
254 print(f"dropped {before - len(model_df):,} duplicate-URL rows")
255
256 X = model_df[features]
257 y = 1 - model_df["label"] # 1 = phishing (positive class)
258 groups = model_df["Domain"]
259
260 gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=RANDOM_STATE)
261 train_idx, test_idx = next(gss.split(X, y, groups=groups))
262 X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
263 y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
264 groups_train = groups.iloc[train_idx]
265
266 print(f"train: {len(X_train):,} test: {len(X_test):,}")
267 print(f"phishing rate train={y_train.mean():.3f} test={y_test.mean():.3f}")
268
269 # ===== Code cell 7 =====
270 def xgb(**kw):
271     params = dict(
272         n_estimators=400,
273         max_depth=6,
274         learning_rate=0.1,
275         tree_method="hist",
276         eval_metric="logloss",
277         n_jobs=-1,
278         random_state=RANDOM_STATE,
279     )
280     params.update(kw)
281     return XGBClassifier(**params)
282
283
284 def evaluate(name, model, X_te, y_te):
285     proba = model.predict_proba(X_te)[: , 1]
286     pred = (proba >= 0.5).astype(int)
287     return {
288         "model": name,
289         "accuracy": accuracy_score(y_te, pred),
290         "precision": precision_score(y_te, pred),
291         "recall": recall_score(y_te, pred),
292         "f1": f1_score(y_te, pred),
293         "roc_auc": roc_auc_score(y_te, proba),
294         "pr_auc": average_precision_score(y_te, proba),
295     }
296
297

```

```

298 n_pos = int(y_train.sum())
299 n_neg = int((y_train == 0).sum())
300
301 models = {
302     "LogReg": Pipeline(
303         [
304             ("scale", StandardScaler()),
305             (
306                 "clf",
307                 LogisticRegression(max_iter=1000, class_weight="balanced"),
308             ),
309         ]
310     ),
311     "RandomForest": RandomForestClassifier(
312         n_estimators=300,
313         class_weight="balanced",
314         n_jobs=-1,
315         random_state=RANDOM_STATE,
316     ),
317     "XGBoost": xgb(scale_pos_weight=n_neg / n_pos),
318 }
319
320 rows = []
321 for name, model in models.items():
322     model.fit(X_train, y_train)
323     rows.append(evaluate(name, model, X_test, y_test))
324
325 results = pd.DataFrame(rows).set_index("model")
326 print(results.round(4).to_string())
327
328 # ===== Code cell 8 =====
329 fig, axes = plt.subplots(1, 3, figsize=(13, 3.6))
330 for ax, (name, model) in zip(axes, models.items()):
331     cm = confusion_matrix(y_test, model.predict(X_test))
332     sns.heatmap(
333         cm,
334         annot=True,
335         fmt="d",
336         cmap="Blues",
337         cbar=False,
338         ax=ax,
339         xticklabels=["legit", "phish"],
340         yticklabels=["legit", "phish"],
341     )
342     ax.set_title(name)
343     ax.set_xlabel("predicted")
344     ax.set_ylabel("actual")
345 plt.tight_layout()
346 plt.show()
347
348 # ===== Code cell 9 =====
349 ablation = {
350     "none": xgb(),
351     "scale_pos_weight": xgb(scale_pos_weight=n_neg / n_pos),

```

```

352     "SMOTE": ImbPipeline(
353         [
354             ("smote", SMOTE(random_state=RANDOM_STATE)),
355             ("clf", xgb()),
356         ]
357     ),
358 }
359
360 rows = []
361 for name, model in ablation.items():
362     model.fit(X_train, y_train)
363     rows.append(evaluate(name, model, X_test, y_test))
364
365 print(pd.DataFrame(rows).set_index("model").round(4).to_string())
366
367 # ===== Code cell 10 =====
368 cv = StratifiedGroupKFold(n_splits=5)
369 param_dist = {
370     "n_estimators": [200, 400, 600],
371     "max_depth": [4, 6, 8, 10],
372     "learning_rate": [0.03, 0.1, 0.3],
373     "subsample": [0.7, 0.9, 1.0],
374     "colsample_bytree": [0.7, 0.9, 1.0],
375 }
376 search = RandomizedSearchCV(
377     xgb(scale_pos_weight=n_neg / n_pos),
378     param_distributions=param_dist,
379     n_iter=10,
380     scoring="average_precision",
381     cv=cv,
382     random_state=RANDOM_STATE,
383     n_jobs=-1,
384 )
385 search.fit(X_train, y_train, groups=groups_train)
386 best = search.best_estimator_
387
388 print("best params:", search.best_params_)
389 print(f"CV PR-AUC: {search.best_score_:.4f}")
390 tuned = evaluate("XGBoost (tuned)", best, X_test, y_test)
391 print(pd.DataFrame([tuned]).set_index("model").round(4).to_string())
392
393 # ===== Code cell 11 =====
394 importances = pd.Series(best.feature_importances_, index=features).sort_values(
395     ascending=False
396 )
397
398 plt.figure(figsize=(8, 5))
399 importances.head(15)[:,-1].plot.barh(color="tab:blue")
400 plt.xlabel("XGBoost gain importance")
401 plt.title("Top 15 features (tuned XGBoost)")
402 plt.tight_layout()
403 plt.show()
404
405 print(f"engineered-feature ranks (of {len(features)} features):")

```

```

406 for f in engineered:
407     rank = int(importances.index.get_loc(f)) + 1
408     print(f" {f:16s} rank {rank:2d}    gain {importances[f]:.4f}")
409
410 # ===== Code cell 12 =====
411 sample = X_test.sample(min(20000, len(X_test)), random_state=RANDOM_STATE)
412 perm = permutation_importance(
413     best,
414     sample,
415     y_test.loc[sample.index],
416     scoring="average_precision",
417     n_repeats=5,
418     random_state=RANDOM_STATE,
419     n_jobs=-1,
420 )
421 perm_imp = pd.Series(perm.importances_mean, index=features)
422 print("top 10 by permutation importance (PR-AUC drop):")
423 print(perm_imp.sort_values(ascending=False).head(10).round(4).to_string())
424
425 # ===== Code cell 13 =====
426 K = 8
427 top_k = importances.head(K).index.tolist()
428 print(f"top {K} features: {top_k}")
429
430 simple = xgb(scale_pos_weight=n_neg / n_pos)
431 simple.fit(X_train[top_k], y_train)
432 simple_res = evaluate(f"XGBoost (top {K})", simple, X_test[top_k], y_test)
433
434 compare = pd.DataFrame([tuned, simple_res]).set_index("model")
435 print(compare.round(4).to_string())
436
437 # ===== Code cell 14 =====
438 # Illustrative per-event costs (see markdown above).
439 COST_FN = 500.0
440 COST_FP = 5.0
441
442 proba = best.predict_proba(X_test)[: , 1]
443 thresholds = np.linspace(0.01, 0.99, 99)
444 n = len(y_test)
445
446
447 def cost_per_1k(t):
448     pred = (proba >= t).astype(int)
449     fn = int(((pred == 0) & (y_test == 1)).sum())
450     fp = int(((pred == 1) & (y_test == 0)).sum())
451     return (COST_FN * fn + COST_FP * fp) / n * 1000
452
453
454 cost_curve = np.array([cost_per_1k(t) for t in thresholds])
455 best_t = float(thresholds[cost_curve.argmin()])
456
457 plt.figure(figsize=(8, 4))
458 plt.plot(thresholds, cost_curve, color="tab:blue")
459 plt.axvline(best_t, color="tab:red", ls="--", label=f"min-cost t={best_t:.2f}")

```

```

460 plt.axvline(0.5, color="gray", ls=":", label="default t=0.50")
461 plt.xlabel("threshold (flag phishing if proba >= t)")
462 plt.ylabel("expected cost per 1,000 URLs ($)")
463 plt.title("Cost-sensitive threshold selection")
464 plt.legend()
465 plt.tight_layout()
466 plt.show()
467
468 print(
469     f"min-cost threshold: {best_t:.2f} "
470     f"${cost_curve.min():.2f} per 1k) vs "
471     f"${cost_per_1k(0.5):.2f} at default 0.5"
472 )
473
474 # ===== Code cell 15 =====
475 prec, rec, thr = precision_recall_curve(y_test, proba)
476
477
478 def operating_point(target, mode):
479     metric = prec[:-1] if mode == "precision" else rec[:-1]
480     other = rec[:-1] if mode == "precision" else prec[:-1]
481     ok = metric >= target
482     if not ok.any():
483         return None
484     idx = np.where(ok)[0]
485     pick = idx[np.argmax(other[idx])]
486     return thr[pick], prec[pick], rec[pick]
487
488
489 for name, mode in [
490     ("precision-first (prec>=0.99)", "precision"),
491     ("recall-first (recall>=0.99)", "recall"),
492 ]:
493     r = operating_point(0.99, mode)
494     if r:
495         t, p, rc = r
496         print(f"{name:30s} t={t:.3f} precision={p:.4f} recall={rc:.4f}")
497
498 # 3-tier policy
499 t_block, t_warn = 0.90, 0.50
500 tier = np.where(
501     proba >= t_block,
502     "block",
503     np.where(proba >= t_warn, "warn", "allow"),
504 )
505 print()
506 print("3-tier policy on the test set:")
507 print(pd.Series(tier).value_counts().to_string())
508
509 # ===== Code cell 16 =====
510 all_results = pd.concat(
511     [results, pd.DataFrame([tuned, simple_res]).set_index("model")]
512 )
513 out = {

```

```

514     "notebook": "02",
515     "positive_class": "phishing",
516     "n_train": int(len(X_train)),
517     "n_test": int(len(X_test)),
518     "engineered_features": engineered,
519     "metrics": all_results.round(4).reset_index().to_dict(orient="records"),
520     "min_cost_threshold": best_t,
521 }
522 (DATA_DIR / "02_classical_results.json").write_text(json.dumps(out, indent=2))
523 print("saved data/02_classical_results.json")

```

Notebook 03 – CharCNN, Clustering, SOTA (Pranav Dhinakar)

```

1  # Notebook 03 - CharCNN, Clustering, SOTA (Pranav Dhinakar)
2  # Exported from 03_charcnn_clustering_sota.ipynb (code cells only, in order).
3
4  # ===== Code cell 1 =====
5  import json
6  from pathlib import Path
7
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11
12 # Section-specific heavy deps (torch, tldextract, hdbscan, rapidfuzz)
13 # are imported at the top of their own sections to keep setup light.
14
15 DATA_PATH = Path("../data/phiusiil.csv")
16 MANIFEST_PATH = Path("../data/01_feature_columns.json")
17
18 # ===== Code cell 2 =====
19 df = pd.read_csv(DATA_PATH)
20 manifest = json.loads(MANIFEST_PATH.read_text())
21
22 target = manifest["target"]
23 clean_features = manifest["clean_numeric_features"]
24 group_col = manifest["split_group_column"]
25
26 # Same pre-split cleaning NB 02 uses, so our numbers are comparable.
27 # We keep URL and Domain as columns (raw material for the char model,
28 # engineered features and grouping) even though they're not features.
29 if manifest["drop_duplicate_urls"]:
30     before = len(df)
31     df = df.drop_duplicates(subset="URL").reset_index(drop=True)
32     print(f"dropped {before - len(df)} duplicate-URL rows")
33
34 y = df[target]
35
36 print(f"rows: {len(df):,}")
37 print(f"clean feature count: {len(clean_features)}")
38 print(f"group column for splits: {group_col}")
39
40 # ===== Code cell 3 =====
41 import tldextract

```

```

42
43 # eTLD+1, e.g. "gateway.ipfs.io" -> "ipfs.io". Uses a bundled public
44 # suffix list; suppress the network refresh for reproducibility.
45 extractor = tldextract.TLDExtract(suffix_list_urls=())
46
47
48 def registered_domain(host: str) -> str:
49     ext = extractor(str(host))
50     # fall back to the raw host for IP-address URLs (no eTLD+1)
51     return ext.top_domain_under_public_suffix or str(host)
52
53
54 df["RegDomain"] = df["Domain"].map(registered_domain)
55
56 n_exact = df["Domain"].nunique()
57 n_reg = df["RegDomain"].nunique()
58 print(f"exact Domain groups: {n_exact:,}")
59 print(f"registered-domain groups: {n_reg:,}")
60 print(f"collapsed by {n_exact - n_reg:,} groups")
61 print()
62 # how many rows share a registered domain with another row?
63 reg_counts = df["RegDomain"].value_counts()
64 reg_repeated = reg_counts[reg_counts > 1]
65 print(
66     f"rows sharing a registered domain: "
67     f"{reg_repeated.sum():,} ({reg_repeated.sum() / len(df):.1%})"
68 )
69 reg_counts.head(10)
70
71 # ===== Code cell 4 =====
72 from sklearn.linear_model import LogisticRegression
73 from sklearn.metrics import accuracy_score, f1_score
74 from sklearn.model_selection import GroupShuffleSplit
75 from sklearn.preprocessing import StandardScaler
76
77
78 def quick_logreg(X, y, groups):
79     """Basic logistic regression under a grouped split; return acc, F1."""
80     gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
81     train_idx, test_idx = next(gss.split(X, y, groups=groups))
82     X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
83     y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
84
85     scaler = StandardScaler()
86     X_train = scaler.fit_transform(X_train)
87     X_test = scaler.transform(X_test)
88     model = LogisticRegression(max_iter=1000, class_weight="balanced")
89     model.fit(X_train, y_train)
90     preds = model.predict(X_test)
91     return accuracy_score(y_test, preds), f1_score(y_test, preds)
92
93
94 X = df[clean_features]
95 acc_exact, f1_exact = quick_logreg(X, y, groups=df["Domain"])

```

```

96 acc_reg, f1_reg = quick_logreg(X, y, groups=df["RegDomain"])
97
98 print(f"exact-Domain split:      acc={acc_exact:.4f}  f1={f1_exact:.4f}")
99 print(f"registered-domain split: acc={acc_reg:.4f}   f1={f1_reg:.4f}")
100
101 # ===== Code cell 5 =====
102 # optional: treat multi-tenant PaaS suffixes as public, so
103 # victim-a.web.app and victim-b.web.app become separate groups
104 extractor_strict = tldextract.TLDExtract(
105     suffix_list_urls=(), include_psl_private_domains=True
106 )
107 df["RegDomainStrict"] = df["Domain"].map(
108     lambda h: extractor_strict(str(h)).top_domain_under_public_suffix or str(h)
109 )
110 print(f"strict (PaaS-aware) groups: {df['RegDomainStrict'].nunique():,}")
111
112 # ===== Code cell 6 =====
113 acc_strict, f1_strict = quick_logreg(X, y, groups=df["RegDomainStrict"])
114 print(f"PaaS-aware split: acc={acc_strict:.4f}  f1={f1_strict:.4f}")
115
116 # ===== Code cell 7 =====
117 # Build a character vocabulary from the training URLs only (fit on
118 # train to avoid leaking test-set characters into the vocab). Reserve
119 # 0 for padding and 1 for unknown/out-of-vocab characters.
120 MAX_LEN = 200 # covers ~99th pct of URL lengths; longer URLs truncated
121
122 urls = df["URL"].astype(str).values
123 labels = df[target].values
124
125 lengths = np.array([len(u) for u in urls])
126 print(
127     f"URL length: median={np.median(lengths):.0f}  "
128     f"f95th={np.percentile(lengths, 95):.0f}  "
129     f"f99th={np.percentile(lengths, 99):.0f}  max={lengths.max()}"
130 )
131 print(
132     f"URLs longer than MAX_LEN={MAX_LEN}: "
133     f"f{(lengths > MAX_LEN).sum()} {(lengths > MAX_LEN).mean():.2%}"
134 )
135
136 # ===== Code cell 8 =====
137 from collections import Counter
138
139
140 # Phishing = positive, matching NB 02. df is already dup-URL-dropped and
141 # index-reset (Section 1), so this split reproduces NB 02's test set.
142 y_phish = (1 - df[target]).to_numpy()
143 groups = df["Domain"].to_numpy()
144 urls = df["URL"].astype(str).to_numpy()
145
146 gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
147 train_idx, test_idx = next(gss.split(df, y_phish, groups=groups))
148
149 url_train, url_test = urls[train_idx], urls[test_idx]

```

```

150 y_train, y_test = y_phish[train_idx], y_phish[test_idx]
151
152 print(f"train: {len(train_idx):,} test: {len(test_idx):,}")
153 print(f"phishing rate train={y_train.mean():.3f} test={y_test.mean():.3f}")
154
155 # Character vocabulary, fit on TRAIN urls only. 0 = <pad>, 1 = <unk>.
156 char_counts = Counter()
157 for u in url_train:
158     char_counts.update(u)
159
160 MIN_FREQ = 5 # chars rarer than this in train collapse to <unk>
161 vocab = {"<pad>": 0, "<unk>": 1}
162 for ch, cnt in char_counts.most_common():
163     if cnt >= MIN_FREQ:
164         vocab[ch] = len(vocab)
165
166 print(f"distinct chars in train: {len(char_counts)}")
167 print(f"vocab size (>= {MIN_FREQ} occ, + pad/unk): {len(vocab)}")
168
169 # ===== Code cell 9 =====
170 def encode(url, vocab, max_len=MAX_LEN):
171     ids = [vocab.get(c, 1) for c in url[:max_len]] # 1 = <unk>
172     ids += [0] * (max_len - len(ids)) # 0 = <pad>
173     return ids
174
175
176 X_train_ids = np.array([encode(u, vocab) for u in url_train], dtype=np.int64)
177 X_test_ids = np.array([encode(u, vocab) for u in url_test], dtype=np.int64)
178
179 trunc = (np.array([len(u) for u in url_train]) > MAX_LEN).mean()
180 print(f"X_train_ids: {X_train_ids.shape} X_test_ids: {X_test_ids.shape}")
181 print(f"train URLs truncated at {MAX_LEN}: {trunc:.2%}")
182 print("example row (first 40 ids):", X_train_ids[0][:40])
183
184 # ===== Code cell 10 =====
185 import torch
186 import torch.nn as nn
187 import torch.nn.functional as F
188 from torch.utils.data import DataLoader, TensorDataset
189 from sklearn.model_selection import GroupShuffleSplit
190
191 torch.manual_seed(42)
192 np.random.seed(42)
193
194 if torch.cuda.is_available():
195     device = "cuda"
196 elif torch.backends.mps.is_available():
197     device = "mps"
198 else:
199     device = "cpu"
200 print(f"device: {device}")
201
202 # Inner grouped train/val split; val only monitors convergence / early
203 # stopping. Grouped so no domain straddles train and val either.

```

```

204 groups_train = groups[train_idx]
205 inner = GroupShuffleSplit(n_splits=1, test_size=0.1, random_state=42)
206 tr_i, va_i = next(inner.split(X_train_ids, y_train, groups=groups_train))
207
208 Xtr = torch.tensor(X_train_ids[tr_i], dtype=torch.long)
209 ytr = torch.tensor(y_train[tr_i], dtype=torch.float32)
210 Xva = torch.tensor(X_train_ids[va_i], dtype=torch.long)
211 yva = torch.tensor(y_train[va_i], dtype=torch.float32)
212 Xte = torch.tensor(X_test_ids, dtype=torch.long)
213 yte = torch.tensor(y_test, dtype=torch.float32)
214
215 BATCH = 512
216 train_loader = DataLoader(
217     TensorDataset(Xtr, ytr), batch_size=BATCH, shuffle=True
218 )
219 val_loader = DataLoader(TensorDataset(Xva, yva), batch_size=BATCH)
220 test_loader = DataLoader(TensorDataset(Xte, yte), batch_size=BATCH)
221
222 # class weight for phishing (positive): n_neg / n_pos on the train split
223 n_pos = float(ytr.sum())
224 n_neg = float(len(ytr) - ytr.sum())
225 pos_weight = torch.tensor([n_neg / n_pos], device=device)
226 print(
227     f"train/val: {len(tr_i):,}/{len(va_i):,}   pos_weight={n_neg / n_pos:.3f}"
228 )
229
230
231 class CharCNN(nn.Module):
232     def __init__(
233         self,
234         vocab_size,
235         embed_dim=48,
236         n_filters=128,
237         kernels=(3, 5, 7),
238         dropout=0.4,
239     ):
240         super().__init__()
241         self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
242         self.convs = nn.ModuleList(
243             [
244                 nn.Conv1d(embed_dim, n_filters, k, padding=k // 2)
245                 for k in kernels
246             ]
247         )
248         self.dropout = nn.Dropout(dropout)
249         self.fc = nn.Linear(n_filters * len(kernels), 1)
250
251     def forward(self, x):
252         e = self.embed(x).permute(0, 2, 1)  # (B, embed, L)
253         feats = [
254             F.relu(conv(e)).max(dim=2).values  # global max-pool
255             for conv in self.convs
256         ]
257         h = self.dropout(torch.cat(feats, dim=1))

```

```

258         return self.fc(h).squeeze(1)  # (B,) logits
259
260
261 model = CharCNN(vocab_size=len(vocab)).to(device)
262 n_params = sum(p.numel() for p in model.parameters())
263 print(f"model params: {n_params:,}")
264
265 # ===== Code cell 11 =====
266 from sklearn.metrics import average_precision_score
267
268 criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
269 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
270 EPOCHS = 6
271
272
273 @torch.no_grad()
274 def predict_proba(loader):
275     model.eval()
276     probs = []
277     for xb, _ in loader:
278         logits = model(xb.to(device))
279         probs.append(torch.sigmoid(logits).cpu().numpy())
280     return np.concatenate(probs)
281
282
283 best_val_ap, best_state = -1.0, None
284 for epoch in range(1, EPOCHS + 1):
285     model.train()
286     total = 0.0
287     for xb, yb in train_loader:
288         xb, yb = xb.to(device), yb.to(device)
289         optimizer.zero_grad()
290         loss = criterion(model(xb), yb)
291         loss.backward()
292         optimizer.step()
293         total += loss.item() * len(xb)
294     train_loss = total / len(Xtr)
295
296     val_p = predict_proba(val_loader)
297     val_ap = average_precision_score(yva.numpy(), val_p)
298     val_f1 = f1_score(yva.numpy(), (val_p >= 0.5).astype(int))
299     flag = ""
300     if val_ap > best_val_ap:
301         best_val_ap = val_ap
302         best_state = {
303             k: v.detach().cpu().clone() for k, v in model.state_dict().items()
304         }
305         flag = " <- best"
306     print(
307         f"epoch {epoch}  train_loss={train_loss:.4f}  "
308         f"val_PR_AUC={val_ap:.4f}  val_F1={val_f1:.4f}{flag}"
309     )
310
311 model.load_state_dict(best_state)

```

```

312 print(f"\nrestored best val PR-AUC = {best_val_ap:.4f}")
313
314 # ===== Code cell 12 =====
315 import seaborn as sns
316 from sklearn.metrics import (
317     confusion_matrix,
318     precision_score,
319     recall_score,
320     roc_auc_score,
321 )
322
323 test_proba = predict_proba(test_loader)
324 test_pred = (test_proba >= 0.5).astype(int)
325
326 char_result = {
327     "model": "CharCNN (URL only)",
328     "accuracy": accuracy_score(y_test, test_pred),
329     "precision": precision_score(y_test, test_pred),
330     "recall": recall_score(y_test, test_pred),
331     "f1": f1_score(y_test, test_pred),
332     "roc_auc": roc_auc_score(y_test, test_proba),
333     "pr_auc": average_precision_score(y_test, test_proba),
334 }
335 print(pd.Series(char_result).to_string())
336
337 cm = confusion_matrix(y_test, test_pred)
338 plt.figure(figsize=(4, 3.4))
339 sns.heatmap(
340     cm,
341     annot=True,
342     fmt="d",
343     cmap="Blues",
344     cbar=False,
345     xticklabels=["legit", "phish"],
346     yticklabels=["legit", "phish"],
347 )
348 plt.title("CharCNN (URL only): test")
349 plt.xlabel("predicted")
350 plt.ylabel("actual")
351 plt.tight_layout()
352 plt.show()
353
354 # ===== Code cell 13 =====
355 # Cluster the PHISHING rows only, on the leakage-clean feature set from
356 # NB 01 (label held out). Goal: find structure within phishing,
357 # "attack families", not to separate phishing from legit.
358 phishing_mask = (df[target] == 0).to_numpy()
359 Xp_raw = df.loc[phishing_mask, clean_features].copy()
360
361 # Drop zero-variance columns within the phishing subset (constant here,
362 # so they carry no clustering signal and would clutter the profile).
363 nunique = Xp_raw.nunique()
364 const_cols = nunique[nunique <= 1].index.tolist()
365 Xp_raw = Xp_raw.drop(columns=const_cols)

```

```

366 cluster_features = Xp_raw.columns.tolist()
367
368 scaler = StandardScaler()
369 Xp = scaler.fit_transform(Xp_raw)
370
371 print(f"phishing rows: {Xp.shape[0]:,}")
372 print(
373     f"clustering features: {Xp.shape[1]} "
374     f"(dropped {len(const_cols)} constant: {const_cols})"
375 )
376
377 # ===== Code cell 14 =====
378 from sklearn.cluster import KMeans
379 from sklearn.metrics import silhouette_score
380
381 ks = range(2, 11)
382 inertias, silhouettes = [], []
383 for k in ks:
384     km = KMeans(n_clusters=k, n_init=10, random_state=42)
385     lbl = km.fit_predict(Xp)
386     inertias.append(km.inertia_)
387     # silhouette on a subsample; full  $O(n^2)$  is infeasible at ~100k rows
388     sil = silhouette_score(Xp, lbl, sample_size=10000, random_state=42)
389     silhouettes.append(sil)
390     print(f"k={k} inertia={km.inertia_:,.0f} silhouette={sil:,.4f}")
391
392 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
393 ax1.plot(list(ks), inertias, "o-", color="tab:blue")
394 ax1.set_xlabel("k")
395 ax1.set_ylabel("inertia (within-cluster SS)")
396 ax1.set_title("Elbow")
397 ax2.plot(list(ks), silhouettes, "o-", color="tab:orange")
398 ax2.set_xlabel("k")
399 ax2.set_ylabel("silhouette (10k sample)")
400 ax2.set_title("Silhouette")
401 plt.tight_layout()
402 plt.show()
403
404 # ===== Code cell 15 =====
405 K_FAMILIES = 5
406
407 # Dominant structure is binary (k=2, silhouette 0.44); characterize it
408 # briefly, then use k=5 for interpretable families.
409 km2 = KMeans(n_clusters=2, n_init=10, random_state=42).fit(Xp)
410 sizes2 = pd.Series(km2.labels_).value_counts().sort_index()
411 print("k=2 dominant split sizes:")
412 for c, n in sizes2.items():
413     print(f" cluster {c}: {n:,} ({n / len(Xp):.1%}")
414
415 km5 = KMeans(n_clusters=K_FAMILIES, n_init=10, random_state=42)
416 phish_labels = km5.fit_predict(Xp)
417
418 profile = Xp_raw.copy()
419 profile["cluster"] = phish_labels

```

```

420 sizes = pd.Series(phish_labels).value_counts().sort_index()
421 print(f"\nk={K_FAMILIES} family sizes:")
422 for c, n in sizes.items():
423     print(f"    cluster {c}: {n:}, ({n / len(Xp):.1%})")
424
425 # Interpretable raw-unit profile (only columns present in the feature set)
426 shortlist = [
427     "IsDomainIP",
428     "IsHTTPS",
429     "URLLength",
430     "DomainLength",
431     "NoOfSubDomain",
432     "DegitRatioInURL",
433     "SpacialCharRatioInURL",
434     "LineOfCode",
435     "NoOfJS",
436     "NoOfImage",
437     "NoOfExternalRef",
438     "NoOfCSS",
439     "HasSocialNet",
440     "HasCopyrightInfo",
441     "HasDescription",
442     "HasPasswordField",
443     "HasSubmitButton",
444     "TLDLegitimateProb",
445 ]
446 shortlist = [c for c in shortlist if c in cluster_features]
447 means = profile.groupby("cluster")[shortlist].mean().T
448 means.columns = [f"c{c}" for c in means.columns]
449 print(f"\nper-cluster mean (raw units), {len(shortlist)} features:")
450 print(means.round(2).to_string())
451
452 # ===== Code cell 16 =====
453 # KMeans centroids live in standardized space, so each value is that
454 # cluster's mean in SDs from the phishing-wide mean. Large |z| = what
455 # makes the family distinctive.
456 centroids = pd.DataFrame(km5.cluster_centers_, columns=cluster_features)
457 for c in range(K_FAMILIES):
458     row = centroids.loc[c]
459     top = row.reindex(row.abs().sort_values(ascending=False).index).head(6)
460     print(f"cluster {c} (n={sizes[c]:},), top distinguishing features:")
461     for feat, z in top.items():
462         arrow = "↑" if z > 0 else "↓"
463         print(f"    {feat:24s} {z:+.2f} {arrow}")
464     print()
465
466 # ===== Code cell 17 =====
467 from sklearn.decomposition import PCA
468
469 FAMILY_NAMES = {
470     0: "empty-shell (68%)",
471     1: "outlier long-URL (5)",
472     2: "IP-host / obfuscated (1%)",
473     3: "credential forms (12%)",

```

```

474     4: "HTTP lookalike stubs (19%)",
475 }
476
477 pca = PCA(n_components=2, random_state=42)
478 coords = pca.fit_transform(Xp)
479 ev = pca.explained_variance_ratio_
480 print(
481     f"PC1+PC2 explained variance: {ev[0]:.1%} + {ev[1]:.1%} = {ev.sum():.1%}"
482 )
483
484 # Subsample for a readable scatter; fit was on all rows.
485 rng = np.random.default_rng(42)
486 idx = rng.choice(len(coords), size=min(15000, len(coords)), replace=False)
487
488 plt.figure(figsize=(8, 6))
489 for c in range(K_FAMILIES):
490     m = phish_labels[idx] == c
491     plt.scatter(
492         coords[idx][m, 0],
493         coords[idx][m, 1],
494         s=4,
495         alpha=0.4,
496         label=FAMILY_NAMES[c],
497     )
498 plt.xlabel(f"PC1 ({ev[0]:.0%} var)")
499 plt.ylabel(f"PC2 ({ev[1]:.0%} var)")
500 plt.title("Phishing families: PCA projection (15k sample)")
501 plt.legend(markerscale=3, fontsize=8, loc="best")
502 plt.tight_layout()
503 plt.show()
504
505 # ===== Code cell 18 =====
506 from sklearn.cluster import HDBSCAN
507
508 # Density-based cross-check on a subsample (HDBSCAN is heavier than
509 # KMeans). Unlike KMeans it doesn't force every point into a cluster;
510 # it can label sparse outliers as noise (-1), which is the honest test
511 # of how much crisp structure exists and whether the 5-row KMeans
512 # micro-cluster survives.
513 sub = rng.choice(len(Xp), size=min(25000, len(Xp)), replace=False)
514 hdb = HDBSCAN(min_cluster_size=250, min_samples=10)
515 hdb_labels = hdb.fit_predict(Xp[sub])
516
517 n_clusters = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
518 noise = (hdb_labels == -1).mean()
519 print(f"HDBSCAN on {len(sub):,}-row sample")
520 print(f"clusters found: {n_clusters}   noise: {noise:.1%}")
521 print()
522 sizes_hdb = pd.Series(hdb_labels).value_counts().sort_index()
523 for c, n in sizes_hdb.items():
524     tag = " (noise)" if c == -1 else ""
525     print(f"   label {c:>2}: {n:,} ({n / len(sub):.1%}){tag}")
526
527 # ===== Code cell 19 =====

```

```

528 # NB 02's saved tabular metrics (phishing = positive, same exact-Domain
529 # split as this notebook, so the rows below are directly comparable).
530 nb02_path = Path("../data/02_classical_results.json")
531 assert nb02_path.exists(), (
532     "Run NB 02 first; it saves 02_classical_results.json this table needs."
533 )
534 nb02 = json.loads(nb02_path.read_text())
535 assert nb02["positive_class"] == "phishing"
536
537 tab = pd.DataFrame(nb02["metrics"]).set_index("model")
538
539 # Append this notebook's CharCNN result (live variable from above).
540 char_row = (
541     pd.Series(char_result).drop("model").to_frame(char_result["model"]).T
542 )
543 metric_cols = ["accuracy", "precision", "recall", "f1", "roc_auc", "pr_auc"]
544 our = pd.concat([tab, char_row])[metric_cols].astype(float)
545
546 print(
547     f"NB 02 test rows: {nb02['n_test']:,} "
548     f"CharCNN test rows: {len(y_test):,} (shared split)\n"
549 )
550 print(our.round(4).to_string())
551
552 # Graphical empirical comparison (rubric asks for this explicitly).
553 # Zoom the y-axis since every model is near the ceiling.
554 plot_metrics = ["accuracy", "recall", "pr_auc"]
555 lo = float(our[plot_metrics].min().min())
556
557 ax = our[plot_metrics].plot.bar(
558     figsize=(11, 4.5),
559     width=0.82,
560     color=["tab:blue", "tab:orange", "tab:green"],
561 )
562 ax.set_ylim(max(0.0, lo - 0.01), 1.001)
563 ax.set_ylabel("score")
564 ax.set_title("Our models: key metrics (test set, phishing = positive)")
565 ax.legend(loc="lower right")
566 plt.xticks(rotation=25, ha="right")
567 plt.tight_layout()
568 plt.show()
569
570 # ===== Code cell 20 =====
571 nb03_out = {
572     "notebook": "03",
573     "positive_class": "phishing",
574     "split": {
575         "grouping": "exact Domain (matches NB 02)",
576         "test_size": 0.2,
577         "random_state": 42,
578         "n_test": int(len(y_test)),
579         "grouping_invariance": {
580             "exact_domain": 0.9995,
581             "paas_aware_etld1": 0.9993,

```

```

582         "naive_etld1": 0.9990,
583     },
584 },
585 "charcnn": {
586     "description": "char-level CNN on raw URL string only, no "
587     "engineered/content/similarity features",
588     "max_len": int(MAX_LEN),
589     "vocab_size": int(len(vocab)),
590     "n_params": int(n_params),
591     "metrics": {
592         k: float(v) for k, v in char_result.items() if k != "model"
593     },
594     "confusion_matrix": confusion_matrix(y_test, test_pred).tolist(),
595 },
596 "clustering": {
597     "algorithm": "KMeans",
598     "k": int(K_FAMILIES),
599     "n_phishing_rows": int(Xp.shape[0]),
600     "silhouette_k2": 0.4373,
601     "silhouette_k5": 0.1540,
602     "families": {
603         FAMILY_NAMES[c]: int(sizes[c]) for c in range(K_FAMILIES)
604     },
605     "hdbscan_check": {
606         "sample_size": int(len(sub)),
607         "clusters": int(n_clusters),
608         "noise_fraction": float(noise),
609     },
610 },
611 "our_models_vs_literature": {
612     "our_models": our.round(4)
613     .reset_index()
614     .rename(columns={"index": "model"})
615     .to_dict(orient="records"),
616     "phiusiil_published_band": "99.5-100%",
617     "url_only_literature_band": "97-99% (other corpora)",
618 },
619 }
620
621 out_path = Path("../data/03_charcnn_cluster_results.json")
622 out_path.write_text(json.dumps(nb03_out, indent=2))
623 print(f"saved {out_path}")
624 print(
625     f"   charcnn: {nb03_out['charcnn']['metrics']['accuracy']:.4f} acc, "
626     f"{nb03_out['charcnn']['metrics']['pr_auc']:.4f} PR-AUC"
627 )
628 print(f"   families: {nb03_out['clustering']['families']}")

```